

课程笔记

课程笔记：[在此填写课程主题]

五道口纳什 & Codex

2026 年 4 月 20 日



视频作者/频道：[在此填写频道或作者]

发布日期：[在此填写发布日期]

视频时长：[在此填写视频时长]

视频链接：未填写

目录

1 开场与背景：从”养虾”到”驯马”的 Agent 热潮	4
1.1 嘉宾介绍与角色定位	4
1.2 春节后 Agent 热度复盘：OpenClaw 的爆火与退潮	4
1.2.1 时间线与关键事件	4
1.2.2 从”春节效应”看中国 AI 认知跃迁	5
1.3 从 OpenClaw 到 Hermes Agent：关注度迁移的拐点	6
1.3.1 一个月周期的热度迁移	6
1.3.2 OpenClaw 的局限性	6
1.3.3 Hermes Agent 如何抓住拐点	7
1.3.4 本章小结	7
2 Hermes Agent 深度解析	7
2.1 什么是 Hermes Agent：定义与核心差异	7
2.1.1 Harness 的定义：模型是大脑，Harness 是双手	7
2.1.2 与 OpenClaw 的三大核心差异	8
2.2 Nous Research 的起源与开源历程	9
2.2.1 从 Discord 频道到顶级开源实验室	9
2.2.2 三大技术贡献	10
2.2.3 Hermes Agent 的”功利主义诞生”	11
2.2.4 惊人的增长数据	11
2.3 成功因素与 MiniMax 合作	12
2.3.1 Hermes Agent 成功的五大因素	12
2.3.2 MiniMax 与 Nous Research 的合作	12
2.4 争议回应：EvoMap 抄袭指控与 M3 预告	13
2.4.1 回应 EvoMap 技术报告指控	13
2.4.2 MiniMax M3 预告	13
2.5 本章小结	14
3 Agent Harness 的技术本质与架构	14
3.1 Harness 的定义：约束与自由的平衡	15
3.1.1 从直觉到定义	15
3.1.2 OpenAI 的正式定义与行业共识	15
3.1.3 Harness 的技术六层	16
3.1.4 同事比喻：从生活直觉理解 Harness	17
3.1.5 高达/机甲比喻：引擎与载具的关系	17
3.1.6 Harness 诞生的物理条件	18
3.2 Multi-Agent：效率与单 Agent 局限性	18
3.2.1 为什么需要 Multi-Agent？	18
3.2.2 效率维度：信息交换的不对称性	18
3.2.3 能力维度：单 Agent 的两大局限	19
3.2.4 Multi-Agent 的最佳实践：Cross-check	21
3.2.5 人类成为瓶颈	21

3.3	Self-Evolution: 自进化的本质	21
3.3.1	一个问题: 为什么 Agent ” 自进化” 被如此推崇?	21
3.3.2	Self-Evolution 的定义	22
3.3.3	效率视角: 人类是瓶颈, 必须减少参与	22
3.3.4	产品视角: 从冷启动到深度磨合	23
3.3.5	关系扭转: 从” 工具使用者” 到” 辅佐者”	23
3.3.6	高达 Wing 的隐喻: Skill、COI 与 Hooks 的沉淀	24
3.4	本章小结	24
4	模型、Agent 与产品: 协同与内化	25
4.1	Skill 蒸馏: 真价值还是幻想?	25
4.1.1	现象: 花叔的爆火与用户的深层诉求	25
4.1.2	反方视角: 被蒸馏的其实是人类	26
4.1.3	价值评估: 为什么 Skill 仍然重要?	26
4.2	MiniMax Agent 的定位与产品哲学	28
4.2.1	爆论: 不以 AGI 为目标的模型公司不应该存在	28
4.2.2	数字员工: 一个有趣的社会实验	28
4.2.3	模型与 Agent 的协同内化循环	29
4.3	OpenClaw 化与产品形态演进	29
4.3.1	Cloud Code 的 “OpenClaw 化”	29
4.3.2	Agent 赛道的随机游走与质变点	30
4.3.3	Skill + COI 范式: 智能的快速扩散	30
4.4	Chat Bot vs Agent: 模型需要多做什么?	30
4.4.1	根本差异: 一次性回答 vs 持续探索	30
4.4.2	In-Life Thinking: 交互后的重新思考	32
4.4.3	Agentic 与 Chat 的最根本差别	32
4.5	本章小结	34
5	中国模型差距、创业机会与行业趋势	35
5.1	中国模型与 OpenAI/Anthropic 的差距	35
5.1.1	差距一: 任务定义的科学方法	36
5.1.2	差距二: AGI 信念与战略定力	36
5.1.3	差距三: GDP 占比的终极标尺	36
5.2	通用 Agent vs 垂直 Agent	37
5.2.1	阿岛: 垂直 Agent 解决最后一公里	37
5.2.2	泽音: 通用 Agent 最终也能做	38
5.2.3	案例交锋: 视频剪辑领域	38
5.3	Agent Infra 创业机会	38
5.3.1	不是创业机会: 身份认证与支付配对	38
5.3.2	有机会: 面向 Agent 的工具与环境	38
5.3.3	云厂商会覆盖: 沙箱、Memory、Runtime	39
5.4	GPT-4.7 与 Anthropic 安全策略评析	39
5.4.1	GPT-4.7: 强 RL 模型的进步与代价	39

5.4.2	Cloud Code 强制实名：两种解读	40
5.4.3	Anthropic Methods 模型不发布：安全还是垄断?	40
5.5	本章小结	41
6	源代码泄露、产品演进与未来畅想	41
6.1	Cloud Code 源代码泄露的影响	41
6.2	Manus 的演进与商业模式变迁	44
6.3	未来 Agent 产品形态畅想	45
6.4	本章小结	47
7	收尾：反共识、人文关怀与寄语	48
7.1	关于 Agent 的反共识观点	48
7.2	对 B 站创作者与观众的寄语	49
7.3	本章小结	51
8	总结与延伸	52
8.1	嘉宾的实质性闭幕讨论	52
8.2	结构化提炼：核心主张与关键机制	53
8.3	跨章节的关联与概念压缩	54
8.4	具体收获、开放问题与下一步思考	54
8.5	对 Agent Harness 本质的最终回答	55

1 开场与背景：从”养虾”到”驯马”的 Agent 热潮

1.1 嘉宾介绍与角色定位

本次深度对谈由主持人 Skyter 主持，邀请到了三位在 Agent 领域具有直接实践经验的核心嘉宾：

- **阿岛**：黑 max（即 Hermes Agent 团队中文名）首席架构师，深度参与 MiniMax M2 系列模型、海螺等产品的研发，拥有多年互联网创业与工作经历。
- **泽音**：MiniMax Agent 研发工程师，负责 MiniMax 官网 Agent、Max Cloud 及 MiniMax Armies 的开发工作。
- **Tommy**（线上连线）：Hermes Agent 产品与策略负责人，代表 Nous Research 团队参与本次对话。

核心背景

Hermes Agent（国内昵称”驯马”）是近期中国热度最高的开源 Agent 之一。本次直播是 Hermes Agent 团队首次与中国用户进行深度交流，也是 MiniMax 与 Nous Research 合作关系的一次公开呈现。

三位嘉宾分别从**模型研发**、**Agent 产品工程**和**开源社区策略**三个维度，构成了理解当前 Agent 热潮的完整视角。阿岛代表”模型 + Agent”的整合视角，泽音代表产品落地与用户交互视角，Tommy 则代表开源社区与全球化运营视角。

1.2 春节后 Agent 热度复盘：OpenClaw 的爆火与退潮

1.2.1 时间线与关键事件

2025 年春节后的一个多月，中国 AI 社区经历了一次前所未有的 Agent 热潮。阿岛将这一过程形容为”急速上升的波峰”，其热度甚至与 MiniMax 的股价波动产生了关联。

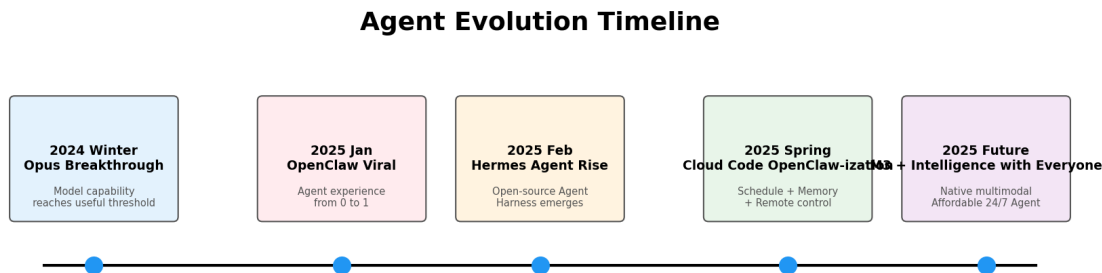


图 1: Agent 热潮时间线：从 OpenClaw 爆火到 Hermes Agent 崛起

1

关键时间线如下：

1. **1 月初**：MiniMax 团队与 OpenClaw 团队（负责人 Peter）建立联系。
2. **IPO 上市当天**：Anthropic 封禁了 OpenClaw 对 Claude 订阅的使用，OpenClaw 急需寻找替代模型。
3. **春节期间**：OpenClaw（国内昵称”龙虾”，使用行为戏称”养虾”）在海外率先走红，随后在国内引发更大规模的热潮。
4. **春节后**：中国用户完成了从”听说过 AI”到”真正使用过 Agent”的跨越。

为什么 OpenClaw 在国内比海外更火？

阿岛和泽音共同分析了这一现象的三层原因：

1. **海外已有成熟体验**：国外用户此前已接触过 Cloud Code、Codex、Coworker 等较为成熟的 Agent 产品，OpenClaw 带来的新鲜感有限。
2. **国内完成”从 0 到 1”跨越**：中国用户此前缺乏好用的 Agent 体验，OpenClaw 通过 IM（即时通讯）这一最熟悉的方式，”捅破了窗户纸”。
3. **字节豆包春晚效应**：字节跳动在春晚期间对豆包的 All-in 投入，极大地提升了全民 AI 认知，为 Agent 热潮奠定了用户基础。

1.2.2 从”春节效应”看中国 AI 认知跃迁

泽音特别指出，中国的农历春节在 AI 传播史上具有特殊的”认知跃迁”效应：

- 2024 年春节后：DeepSeek 爆火，让大众第一次意识到”中国也能做出顶尖 AI”。

¹视频画面时间区间：00:02:00-00:07:00。

- 2025 年春节后：OpenClaw 爆火，让大众第一次真正**使用**了 Agent，进入了”Agent 时代”。

这种春节效应的本质是：假期提供了充足的闲暇时间，而家庭聚会场景加速了技术产品的口碑传播。当用户发现身边的亲戚朋友都在”养虾”时，Agent 就从极客玩具变成了全民话题。

1.3 从 OpenClaw 到 Hermes Agent：关注度迁移的拐点

1.3.1 一个月周期的热度迁移

泽音观察到一个清晰的热度迁移周期，约一个月：

OpenClaw 爆火 → 热度平稳 → Hermes Agent（Armies）火起



图 2: Hermes Agent 崛起：关注度迁移的关键拐点

2

这一迁移并非偶然，而是基于用户痛点的自然演进。

1.3.2 OpenClaw 的局限性

在大量用户体验后，OpenClaw 的若干结构性局限逐渐暴露：

OpenClaw 的核心痛点

1. **软件特性导致不稳定**：作为早期产品，其架构在长时间运行和高并发场景下存在可靠性问题。
2. **记忆刷新机制**：每天凌晨 4 点刷新记忆，用户需要重新建立与 Agent 的关系和上下文。这意味着前一天辛苦培养的”默契”会在一夜之间归零。

这一记忆机制的问题尤为关键。Agent 的核心价值在于**持续理解用户**，而每日记忆刷新直接破坏了这一价值主张。用户投入时间训练 Agent，却得不到累积性的回报——这在产品体验上是致命的。

²视频画面时间区间：00:07:00–00:11:00。

1.3.3 Hermes Agent 如何抓住拐点

Hermes Agent 的崛起正是建立在对上述痛点的精准解决之上：

Hermes Agent 的差异化优势

- **多层次记忆系统**：泽音特别强调，Hermes Agent 在记忆方面”做了非常多层次的记忆”，从根本上解决了 OpenClaw 的”每日失忆”问题。
- **双向奔赴**：一方面模型公司（如 MiniMax）做出了更好的 Agent；另一方面，经过 OpenClaw 教育后的用户，对 AI 的诉求和接受度也显著提高。

这种关注度迁移的本质是**用户认知的升级**：OpenClaw 完成了 Agent 的”市场教育”，让用户第一次体验到”AI 可以帮我做事”；而 Hermes Agent 则在此基础上提供了更稳定、更持久、更可信赖的体验，满足了用户被拉高后的期待。

1.3.4 本章小结

第 1 章核心要点

1. **Agent 热潮的时间线**：2025 年春节后，OpenClaw 引发了中国第一次全民 Agent 体验浪潮。
2. **国内更火的原因**：海外已有渐进式体验，国内完成”从 0 到 1”跨越；字节豆包春晚投入提供了认知基础。
3. **热度迁移逻辑**：约一个月周期内，OpenClaw 的局限性（尤其是记忆刷新问题）催生了 Hermes Agent 的崛起。
4. **核心概念**：**Agent**——能自主感知环境、做出决策并执行动作的 AI 系统；**OpenClaw**——引发本次热潮的标志性 Agent 产品。

2 Hermes Agent 深度解析

2.1 什么是 Hermes Agent：定义与核心差异

2.1.1 Harness 的定义：模型是大脑，Harness 是双手

Tommy 对 Hermes Agent 给出了一个简洁而精准的定义：

Hermes Agent 的定义

Hermes Agent = 开源 Agent Harness

其中，**Harness**（马具/ Harness）是让模型在真实世界中操作的框架。Tommy 用了一个形象的比喻：

”如果模型是大脑（brain），那么 Agent Harness 就是双手（hands）。”

Agent Harness Architecture

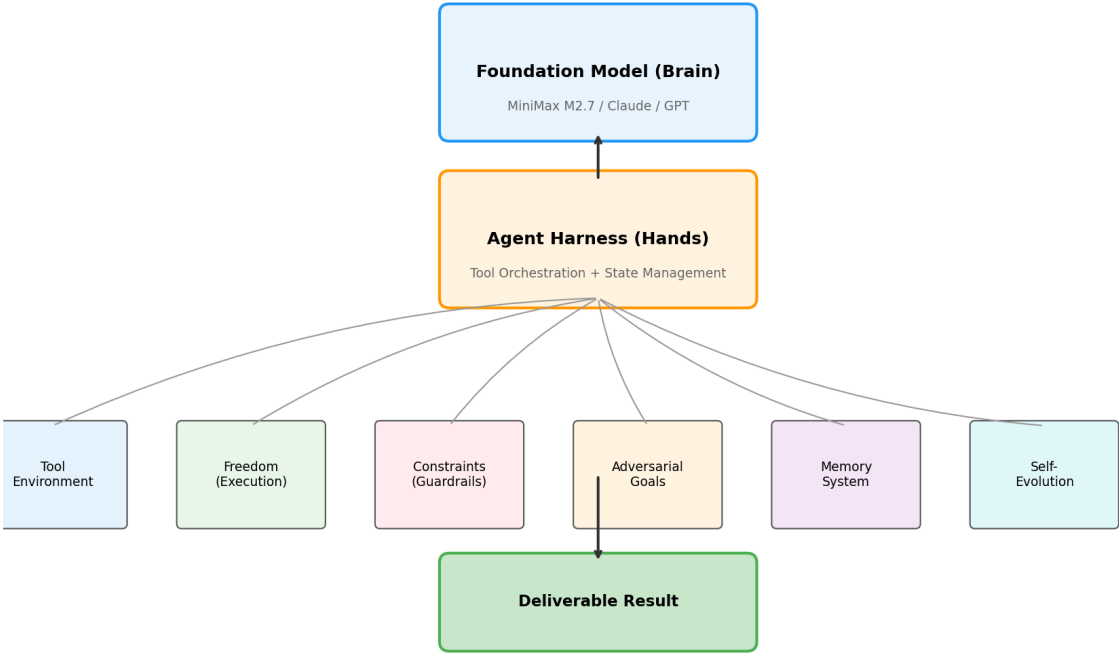


图 3: Harness 架构：模型 = 大脑，Harness = 双手

Harness 的核心功能包括：

- **工具编排**（Tool Orchestration）：管理模型可以调用的各类工具。
- **编排循环**（Orchestration Loop）：控制模型思考-行动-观察的循环流程。
- **状态管理**（State Management）：维护 Agent 运行过程中的上下文和状态。
- **错误处理**（Error Handling）：捕获并处理模型在执行过程中遇到的异常。

2.1.2 与 OpenClaw 的三大核心差异

Tommy 明确指出了 Hermes Agent 与 OpenClaw 的三个关键差异：

1. 易用性（Usability）：

- 从安装到完成 Git 工作，仅需 2 分钟或更短。
- 无代码臃肿（No Code Bloat），无需 tinkering 大量设置。
- 用户可以直接 drop 到本地电脑或 VPS 上快速运行。

2. 记忆系统（Memory）——最有趣的组件：

- 传统 Agent 的痛点：第一次做对了，第二次做同样的事却出错，导致用户难以信任。
- Hermes Agent 的解决方案：记住工作流，保存为 **Skill**，确保可复现、可信任。

³视频画面时间区间：00:11:00-00:14:00。

- 一旦 Agent 学会做某件事，就能始终复现该路径。

3. 自进化 (Self-Evolution):

- 通过知识压缩 (Knowledge Compression) 实现持续改进。
- 带来跨模型一致性: 8 个不同模型 + 同一 Harness + 同一 Skills = 相同输出。
- 允许用户为不同任务切换模型，同时保持核心任务的高性能和可信赖性。



图 4: 记忆系统: Memory 到 Skill 的沉淀与跨模型一致性

4

Skill 的定义

Skill 是 Agent 可学习、保存、复用的 workflow 或能力单元。当 Agent 成功完成某项任务后，该任务的执行路径被沉淀为 Skill，后续遇到相同或相似任务时可直接调用，无需重新学习。

2.2 Nous Research 的起源与开源历程

2.2.1 从 Discord 频道到顶级开源实验室

Tommy 介绍了 Nous Research 的独特起源故事：

”Nous Research 始于 2022 年的一个 Discord 频道。一群热爱开源、热爱玩 AI 模型的人聚集在一起，这个频道至今仍是我们的核心沟通渠道。”

⁴视频画面时间区间：00:13:00-00:15:00。



图 5: Nous Research: 从 Discord 社区到 Hermes Agent 的开源历程

5

2.2.2 三大技术贡献

Nous Research 在 Hermes Agent 爆火之前，已经积累了多项被业界广泛认可的研究成果：

1. Hermes 系列后训练 (Llama 时代)：

- 在 Meta 发布 Llama 模型后，Nous 进行了高质量的后训练 (Post-Training)。
- **Hermes** 名称源自希腊神话中的信使神，寓意“传递信息”。
- 目标是让模型听起来更人性化，而非刻板的 AI 助手口吻，具备更高的可塑性 (Pliability) 和文本多样性。

2. YaRN 算法——上下文长度扩展的革命：

- 将模型的上下文长度从 4K 扩展到 128K。
- 被所有主要模型公司立即采纳。
- 成为“思考模型” (Thinking Models) 的技术基础，进而成为 Agentic 模型的基石。

3. Destroo 论文——分布式训练优化器：

- 一种新型优化器，允许在非共置 GPU 上进行训练。
- 能够聚合分散的算力 (Disparate Compute)，形成有意义的训练运行。
- 对开源模型社区至关重要——当没有单一公司拥有足够算力时，Destroo 让“众筹算力训练”成为可能。

YaRN 与 Destroo 的意义

YaRN 和 Destroo 不仅是技术论文，更代表了 Nous Research 的**开源精神**：做出真正有用的研究，并让它被整个社区采用。这种 track record 也是后续回应抄袭指控时的重要底气。

⁵ 视频画面时间区间：00:15:00–00:18:00。

2.2.3 Hermes Agent 的” 功利主义诞生”

Hermes Agent 的起源与大多数商业产品截然不同：

Hermes Agent 的诞生故事

- **创始人：**Technium（Nous Research 联合创始人、后训练负责人）。
- **初衷：**为自己构建一个助理工具，管理日常工作、后训练任务。
- **时间：**约一年前（即 2024 年初）开始开发。
- **理念：**没有 benchmark 目标，没有商业计划，纯粹为了” 让它为我工作”。
- **开源：**作为开源实验室，自然选择开源；意外获得了远超预期的关注。

这种” 为自己而做” 的起点，恰恰是 Hermes Agent 产品力的根源——它不是为了在评测榜上拿高分，而是为了解决真实的日常问题。

2.2.4 惊人的增长数据

Tommy 分享了 Hermes Agent 在 OpenRouter 平台上的 token 使用量增长：

20 亿 token/天 → 200 亿 token/天 → 近 3000 亿 token/天



图 6: OpenRouter 使用量爆发：从 20 亿到 3000 亿 token

6

这一增长速度在开源项目中极为罕见，也印证了 Agent Harness 这一产品形态的市场需求。

⁶视频画面时间区间：00:18:00-00:20:00。

2.3 成功因素与 MiniMax 合作

2.3.1 Hermes Agent 成功的五大因素

Tommy 总结了 Hermes Agent 快速崛起的多重因素：

1. **模型能力达到有用门槛**：Opus 在冬季的大幅提升是” step function improvement”（阶跃式提升），为 Agent Harness 打开了大门。
2. **开源实验室快速追赶**：开源模型迅速接近 Opus 质量，让用户有了更多选择。
3. **优化可用性而非 benchmark**：将北极星指标定为” 用户任务完成质量”，而非评测分数。
4. **记忆系统建立信任**：解决了 Agent 最核心的” 不可信赖” 问题。
5. **品牌与社区建设**：赛博朋克美学（Cyberpunk Aesthetic）+ ethos 对齐的社区，形成了强烈的文化认同。

开源产品的” 代码臃肿” 陷阱

Tommy 特别警告了开源社区的一个常见陷阱：当产品获得大量关注后，每天收到成百上千的 PR，很容易为了迎合各种需求而导致” 代码臃肿”（Code Bloat），最终损害产品质量。Hermes Agent 团队在这方面保持了高度克制，始终守护产品的易用性。

2.3.2 MiniMax 与 Nous Research 的合作

双方的合作始于一次极为简单的互动：

”Leanna（MiniMax 团队成员）在 Twitter 上 DM 了 Technium，说’ 喜欢你们在做的事情’。”



图 7: MiniMax × Nous Research: 模型公司 + 开源社区的合作范式

7

⁷ 视频画面时间区间：00:22:00–00:25:00。

合作内容涵盖三个层面：

1. **联合 benchmark**：共同评估 Hermes Agent 在不同模型上的表现。
2. **技术方案讨论**：探讨最佳的技术实现路径。
3. **下一代 Agentic 模型训练集成**：尤其是自改进层（Self-Improvement Layer）的深度集成。

关键数据：M2.7 是 Hermes Agent 用户使用量最大的模型

MiniMax M2.7 在 Hermes Agent 用户中的日 token 消耗量超过 **250 亿**，是所有接入模型中最高的。这不仅证明了 M2.7 的模型质量，也体现了双方技术集成的深度。

Tommy 还指出，中国目前在开源模型领域“clearly holds the lead”（明显领先），开源与闭源模型之间的差距从未如此之小，且这一差距还将继续缩小。

2.4 争议回应：EvoMap 抄袭指控与 M3 预告

2.4.1 回应 EvoMap 技术报告指控

在直播过程中，主持人 Skyter 代表观众向 Tommy 提出了一个尖锐问题：近期中国开源项目 EvoMap 发布技术报告，指控 Hermes Agent 的自进化架构与其开源项目 Evolver”几乎完全相同”。

Tommy 的回应清晰而有力：

Tommy 的三点回应

1. **时间线**：Hermes Agent 仓库已存在约一年，Technium 此前从未听说过 EvoMap，直到相关推文出现。
2. **Track Record**：Nous Research 拥有 YaRN、Destroo 等被广泛认可的原创研究，团队“deeply ideologically driven”（深度理念驱动），致力于开源，“certainly aren't copying repos”（绝不会抄袭仓库）。
3. **社区认知**：像 MiniMax 这样深入了解 Nous Research 的团队，都清楚其原创性和开源精神。

这一回应的核心逻辑是**长期 track record 胜过单次指控**——一个在 AI 领域持续产出原创研究、被所有主流模型公司采纳的团队，其原创性是不言自明的。

2.4.2 MiniMax M3 预告

在英文对谈的最后，阿岛向 Tommy 预告了 MiniMax M3 的关键信息：

MiniMax M3 预告要点

1. **规模更大、更智能**：在编码和通用工作场景上都会有显著提升。
2. **原生多模态**：支持视频（Video）和图像（Image）输入。
3. **目标愿景**：让所有人从事创造性工作，重复性工作留给 Agent。
4. **可负担性**：**Intelligence with Everyone**——让每个人都能负担得起 7×24 小时的 Agent。

阿岛特别强调，M3 的发布时间”won’t be far”（不会太远），并邀请观众”please looking forward to it”。

M3 的战略意义

M3 将原生支持视频和图像输入，这意味着 Agent 将能够”看懂”屏幕、视频内容，而不仅仅是处理文本。这将为 Agent 解锁大量全新场景，也是 Hermes Agent 与 MiniMax 深度合作的重要技术基础。

2.5 本章小结

第 2 章核心要点

1. **Harness 的定义**：Agent Harness 是让模型在真实世界操作的框架，模型是大脑，Harness 是双手。核心功能包括工具编排、编排循环、状态管理和错误处理。
2. **Hermes Agent 的三大差异**：易用性（2 分钟上手）、记忆系统（Skill 沉淀与复现）、自进化（跨模型一致性）。
3. **Nous Research 的开源历程**：2022 年从 Discord 频道起步，先后产出 Hermes 后训练系列、YaRN（上下文扩展）、Destroo（分布式训练）等原创研究。
4. **Hermes Agent 的诞生**：Technium 为自己构建的助理工具，约一年前开始，自然开源后意外爆火，OpenRouter token 量从 20 亿/天增至近 3000 亿/天。
5. **MiniMax 合作**：始于 Twitter DM，涵盖联合 benchmark、技术方案讨论和下一代 Agentic 模型训练集成；M2.7 是 Hermes Agent 用户使用量最大的模型（250 亿 + token/天）。
6. **争议回应**：Hermes Agent 仓库已存在约一年，Nous Research 的长期 track record 证明其原创性。
7. **M3 预告**：规模更大、原生多模态（视频 + 图像）、目标 7×24 小时可负担的 Agent。
8. **核心概念**：Agent Harness、Skill、Self-Evolution、Nous Research、YaRN、Destroo、Technium、OpenRouter。

3 Agent Harness 的技术本质与架构

时间边界：00:34:00 – 00:52:00

本章核心目标

通过本章的学习，读者将能够：

1. 准确理解 **Agent Harness** 的技术定义与六层架构；
2. 掌握 **Multi-Agent** 协作的必要性、效率优势与最佳实践；
3. 深入理解 **Self-Evolution（自进化）** 的本质——它为何在 Agent 领域被赋予如此高的期待，以及它与推荐系统“越用越准”的根本差异。

3.1 Harness 的定义：约束与自由的平衡

3.1.1 从直觉到定义

当我们第一次听到“Harness”这个词时，很容易联想到马具——一种用来驾驭烈马、使其按照骑手意图奔跑，同时又不扼杀其力量的装置。这个比喻非常贴切，因为它恰好捕捉了 Agent Harness 的核心矛盾：**既要约束，又要释放。**

Harness 的直觉定义

Agent Harness 是一套**约束 Agent 但给予一定自由**的机制，使其能够在有限的自主空间内，**完整交付高质量成果**。它不是对模型能力的削弱，而是对模型能力的**定向引导**。

泽音在直播中给出了一个更生活化的定义：

“它其实就是一种你去约束 Agent，但是又给他一定自由，同时的话他就能够完整地交付给你一些成果的这样一套玩法。”

这个定义的精妙之处在于它同时包含了三个要素：

- **约束 (Constraint)**：划定边界，防止 Agent 越界或失控；
- **自由 (Freedom)**：在边界内给予充分的操作空间；
- **交付 (Delivery)**：最终目标是产出可验证、可交付的结果，而非仅仅给出建议。

3.1.2 OpenAI 的正式定义与行业共识

Harness 概念的“破圈”，很大程度上归功于 OpenAI 官方发布的一篇文章。在那篇文章中，OpenAI 正式将 Harness 定义为 Agent 的一种应用方式。阿岛补充道：

“我觉得 Harness 这个词是 OpenAI 可能通过那篇文章来定义出来的，但在那段时间其实它已经很火了。我觉得这层窗户纸，它可能是通过那篇文章来捅破的——大家形成了一个共识：我们用什么词来形容我们在做的一切。”

这揭示了一个重要的行业现象：**技术实践往往先于术语共识**。在 OpenAI 文章发布之前，无论是 MiniMax 内部的 Agent 工作流，还是 Hermes Agent 的架构设计，都已经大量实践了 Harness 的思想。术语的统一起到了“捅破窗户纸”的作用，让分散的实践获得了共同的表达框架。

3.1.3 Harness 的技术六层

从偏技术的角度，泽音将 Harness 的架构概括为六个层次。为了便于理解，我们可以将其组织为一个从”环境”到”对抗”的递进结构：



图 8: Harness 六层技术栈：从模型到交付的完整链路

8

1. 工具环境 (Tool Environment)

这是 Harness 的最底层，为 Agent 提供可操作的外部世界。类比给同事一台笔记本电脑、一个电话、一个邮箱账号——Agent 需要这些”基础设施”才能开始工作。工具环境包括但不限于：文件系统访问、代码执行环境、网络请求能力、数据库接口、第三方 API（如邮件服务、部署平台）等。

2. 自由度 (Degree of Freedom)

在工具环境之上，Harness 需要明确 Agent 可以做什么、不能做什么。自由度不是越大越好，也不是越小越好，而是一个需要精心设计的参数。过高的自由度会导致 Agent 行为不可预测；过低的自由度则会束缚 Agent 的创造力，使其沦为简单的命令执行器。

3. 约束 (Constraints)

约束是对自由度的补充和制衡。它可以是硬约束（如”禁止删除生产环境数据库”）或软约束（如”优先使用 Python 而非 Bash 脚本”）。约束的本质是**将人类的经验和安全要求编码到系统中**，让 Agent 在探索时不会触碰红线。

4. 互相对抗性目标 (Adversarial Objectives)

这是 Harness 最精妙的一层。与其让单个 Agent 独自完成任务，不如引入多个 Agent，赋予它们不同甚至相互冲突的目标。例如：一个 Agent 负责推进任务、生成方案；另一个 Agent 负责审查方案、发现问题。这种**对抗性设计**模拟了人类团队中的互相监督机制，是 Harness 产出高质量结果的关键。

⁸视频画面时间区间：00:34:00-00:37:00。

5. 状态管理 (State Management)

长程任务中，Agent 需要记住自己做过什么、当前处于什么阶段、下一步该做什么。状态管理确保任务不会因“失忆”而中断或重复。

6. 编排循环 (Orchestration Loop)

最顶层是编排循环——它决定何时调用哪个 Agent、何时切换任务阶段、何时请求人类介入。编排循环是 Harness 的“指挥家”。

Harness 六层记忆口诀

工具环境 → 自由度 → 约束 → 对抗目标 → 状态管理 → 编排循环

简记为：“工自约对状编”——一个 Harness 从给工具开始，到编排结束。

3.1.4 同事比喻：从生活直理解解 Harness

泽音用了一个极为生动的“同事比喻”来解释 Harness 的完整运作逻辑：

“你就假设一下你有一个同事，然后他全都听你的。但是你得和他先约定好他能干什么、你能干什么。他有些事情不能帮你做，但是有些事情你就完全可以放任他去做。当你约定好这些，并且给他配置好一些——比如说它的模型是什么、它有什么工具等等——给他一定的能力。就像是你给一个同事一台笔记本、一个电话、一个邮箱账号。给他一定能力之后，他就可以开始干活了。”

但单个同事干活，你“肯定不放心”。于是 Harness 引入了下一步：

“这时候你可以去引入多位同事，他们之间互相监督，形成一个制约或者对抗的感觉。在这种基础之上，他们能够形成一个互相监督并且完成任务并最终交付的这样一整套过程。”

这个比喻的价值在于，它将抽象的六层架构映射到了每个人都能理解的工作场景中：

- 给笔记本/电话/邮箱 = 工具环境 + 自由度；
- 约定能做什么 = 约束；
- 多位同事互相监督 = 对抗性目标；
- 完成任务并交付 = 状态管理 + 编排循环的最终输出。

3.1.5 高达/机甲比喻：引擎与载具的关系

阿岛则从另一个角度——高达 (Gundam) 和机甲——来比喻 Harness：

“你有很强大的引擎，我们其实需要去造一个高达或者那样的一个机甲，然后怎么让这个引擎的能力最大地发挥出来。所以我认为 Harness 就是那样一个东西。他们为什么把它叫 Harness？就是你有一匹很厉害的烈马，但怎么样让他最能够发挥出来，就是构造马具——那样的一个过程。”

两个比喻的互补性

- **同事比喻**强调 Harness 的**社会性**——Agent 之间的协作、监督、分工；
- **机甲比喻**强调 Harness 的**技术性**——如何将强大的模型”引擎”封装为可操控的”载具”。

两个比喻合在一起，构成了对 Harness 最完整的直觉理解。

3.1.6 Harness 诞生的物理条件

Harness 并非凭空出现，它的诞生需要三个”物理条件”：

1. **模型能力达到门槛**：模型必须具备基本的 Agentic 能力，能够理解复杂指令、进行多步推理、调用工具。
2. **用户信任的建立**：人们愿意把更多能力开放给 Agent——操作邮箱、部署服务、分享财务数据。这种信任来自于前期 Agent 产品（如 OpenClaw）带来的正向体验。
3. **收益的正向循环**：当用户从 Agent 中获得实际收益后，更愿意投入时间和资源去优化自己的 Agent 工作流，从而推动 Harness 生态的繁荣。

”有这样子人们去放下了对 Agent 的一些怀疑，并且大家又从 Agent 中获得一些收益了以后，然后 Harness 就自然而然地诞生了。”

3.2 Multi-Agent：效率与单 Agent 局限性

3.2.1 为什么需要 Multi-Agent？

在 Harness 的框架中，Multi-Agent（多智能体协作）不是锦上添花，而是**架构设计的核心支柱**。但这也引发了一个自然的质疑：今天的模型智能已经非常高，为什么非得搞一个”Agent Team”出来？让单个模型完成所有工作，不是更简单、更高效吗？

泽音从两个维度回应了这个质疑：**效率维度**和**能力维度**。

3.2.2 效率维度：信息交换的不对称性

泽音提出了一个极具洞察力的思想实验：

”你和模型对话的过程中，其实你产出的信息量是较少的，而模型产出的信息量是较多的。假设有两个模型，它们之间能够以更加高效的效率去交换信息。模型给你发的消息是一个小作文，那么你给模型的可能就两个字——’批准，接着干’。但是如果让两个模型它们之间互相交流，他们的信息量是极大的。在这种极高密度的信息交换的情况下，整体效率就会比人和一个 Agent 进行交互要高得多。”

信息交换效率公式

我们可以用一个简化的模型来理解这种效率差异：

设人与单个 Agent 交互时，人每轮输入的信息量为 I_{human} ，Agent 输出的信息量为 I_{agent} 。通常有：

$$I_{human} \ll I_{agent}$$

这意味着大量的 Agent 输出”浪费”在了等待人类稀疏的反馈上。而两个 Agent 之间交互时：

$$I_{agent_1} \approx I_{agent_2}$$

信息密度匹配，通信带宽被充分利用。Multi-Agent 系统的有效信息吞吐量远高于人-Agent 系统。

Multi-Agent System: Cross-Check Pattern

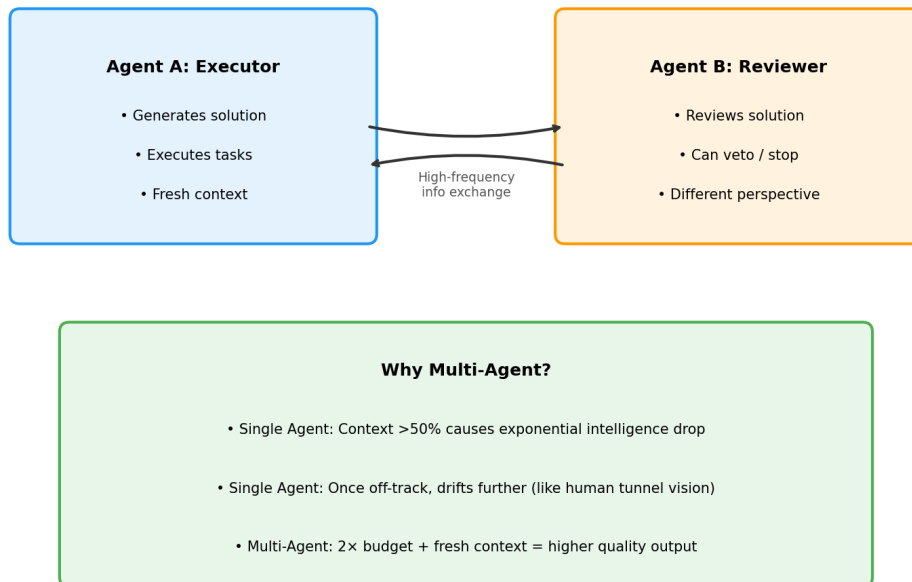


图 9: Multi-Agent Cross-check 模式：对抗视角与全新上下文

9

3.2.3 能力维度：单 Agent 的两大局限

即使不考虑效率，单 Agent 在能力层面也存在两个根本性的局限。

局限一：上下文”降智” 泽音引用了一个已被论证的现象：

” 当你的上下文空间——也就是你和他对话的文本数量——超过了 50% 以后，其实它的

⁹ 视频画面时间区间：00:40:00–00:43:00。

智能水平的下降是指数级下降的。也就是说在单个 Agent 里，你永远没办法和他一直聊下去。”

上下文降智曲线

设模型的上下文窗口总长度为 C_{max} ，当前已使用的上下文长度为 C_{used} 。当：

$$\frac{C_{used}}{C_{max}} > 0.5$$

模型的有效智能 $I_{effective}$ 随已用上下文比例呈近似指数衰减：

$$I_{effective} \approx I_0 \cdot e^{-\lambda \cdot (C_{used}/C_{max} - 0.5)}$$

其中 λ 为衰减系数，与模型架构和训练方式有关。这意味着即使是最强的模型（如 GPT-4.7），在长时间连续使用后也会出现”降智”行为。



图 10: 上下文超过 50% 后的智能衰减：Multi-Agent 的价值所在

10

局限二：长程任务中的”钻牛角尖” 阿岛补充了另一个关键发现：

”最近有一篇论文研究了非常长程的任务。他们发现：如果 Agent 做对了，对的路线一直进行下去，很容易很顺利就做完了。但他只要在某一个地方可能偏了，很容易他就会越来越偏。这其实和人也是类似的——有时候我们去做一件事情，是有可能钻牛角尖的。”

这个发现揭示了一个深层问题：LLM 的训练数据分布偏向于**线性执行轨迹**，而较少包含**反复验证和思考**的模式。因此，当 Agent 在长程任务中偏离正确路径时，它缺乏”自我纠正”的内在机制，反而会在错误方向上越陷越深。

¹⁰ 视频画面时间区间：00:42:00–00:45:00。

3.2.4 Multi-Agent 的最佳实践：Cross-check

面对上述局限，Multi-Agent 提供了系统性的解决方案。阿岛分享了 MiniMax 内部的最佳实践：

Cross-check 机制

Cross-check 是指让两个或多个 Agent 互相审查、验证对方产出的机制。具体实现方式包括：

1. **方案阶段 Cross-check**：Agent A 提出方案，Agent B 从不同角度重新审视，发现潜在问题；
2. **执行阶段 Cross-check**：Agent A 负责执行，Agent B 负责审查并有权“叫停”；
3. **全新上下文重启**：Agent B 不继承 Agent A 的上下文，以“两倍预算 + 全新视角”重新思考。

“如果你设置两个 Agent 的话，它可以没有之前的心智负担，换一个角度重新思考。相当于你有两倍的预算，而且是全新的上下文去思考，往往能达到一个更高的质量。”

这种设计巧妙地规避了单 Agent 的两大局限：

- 通过**切换 Agent**，重置上下文，避免“降智”；
- 通过**引入对抗视角**，打破“钻牛角尖”的恶性循环。

3.2.5 人类成为瓶颈

阿岛还揭示了一个更深层的动机——**人类本身成为了瓶颈**：

“大概在去年 9、10 月份的时候，我自己的工作流中实际上已经不怎么用 IDE。我可能同时并发五六个 Agent，甚至只是在本地，可能在云端有十个 Agent 在为我工作。他们就在 GitHub 上尝试不同的分支和想法。这时候我会发现我成为一个极大的瓶颈——人类成为瓶颈。”

瓶颈转移定律

在 Agent 能力较弱时，**Agent 是瓶颈**——它无法完成复杂任务；
在 Agent 能力增强后，**人类是瓶颈**——人需要在多个 Agent 的上下文之间切换、输入、审批；
解决之道：**让 Agent 之间的协作自动化**，减少人类在循环中的介入频率。

3.3 Self-Evolution: 自进化的本质

3.3.1 一个问题：为什么 Agent “自进化”被如此推崇？

主持人提出了一个尖锐的对比问题：

“我们今天刷抖音也是越刷算法越准，但是我们好像也并没有说抖音会‘自我进化’。但今天为什么在 Agent 领域，‘自进化’就被大家认为是一个那么厉害的词呢？”

这个问题直指 Self-Evolution 概念的核心。要回答它，我们需要先理解 Self-Evolution 在 Agent 语境下的**真正含义**。

3.3.2 Self-Evolution 的定义

Self-Evolution 的形式化定义

Self-Evolution (自进化) 是指 Agent 在减少人类参与的前提下, 通过与真实环境的交互获得反馈, 并将反馈沉淀为可复用的知识 (Skill、COI、Hooks 等), 从而在下一次任务中表现得更好的能力。

与推荐系统的“越刷越准”相比, Agent 的 Self-Evolution 有三个根本差异:

1. **反馈来源不同**: 推荐系统的反馈是“点击/停留”等被动信号; Agent 的反馈是“程序能否运行”“测试结果是否正确”等主动验证;
2. **知识沉淀形式不同**: 推荐系统的“进化”藏在模型权重中, 不可见、不可迁移; Agent 的进化沉淀为 **Skill**、**COI**、**Hooks**, 可被保存、分享、复用;
3. **人类角色不同**: 推荐系统的人类是被动的“数据提供者”; Agent Self-Evolution 中的人类是**方向的指引者**, 保留判断和品味 (Taste)。

3.3.3 效率视角: 人类是瓶颈, 必须减少参与

阿岛从**训练效率**的角度解释了 Self-Evolution 的必然性:

“你会发现, 在这个迭代的过程中, 人开始成为瓶颈。而且人的数量越多, 他的组织效率就是指数级下降的。我经历过非常多的组织都是这样的。要做大模型这样的一次训练数以亿计美元成本的事情, 你需要保持非常高的人才密度和非常精干的组织。那大家想到的一个最有效的方法就是: 减少人的参与, 更多的让 AI 来做。”

他以 MiniMax M2.7 的训练为例:

“在我们的 RL pipeline 里面, 已经是 70%–80% 以上的工作由模型加 Agent 自己来完成。人类工程师只有 7×24 个小时, 我们还要休息。现在剩下的 30% 还要有人去做的工作, 更多的是说——人的判断和 Taste 是非常重要的。”

人类剩余 30% 的工作内容

在 Self-Evolution 的闭环中, 人类不再负责:

- 排查具体实验中的某个特定过程是否出错 (Agent 能查出);
- 在多个上下文中切换和输入 (Agent 之间自动协作);
- 重复性的验证和测试 (Agent 自动执行)。

人类保留的核心价值:

- **判断 (Judgment)**: 实验结果与预期方向是否一致;
- **品味 (Taste)**: 什么方向值得继续探索, 什么应该放弃;
- **创造力 (Creativity)**: 提出新的问题、新的假设、新的方向。

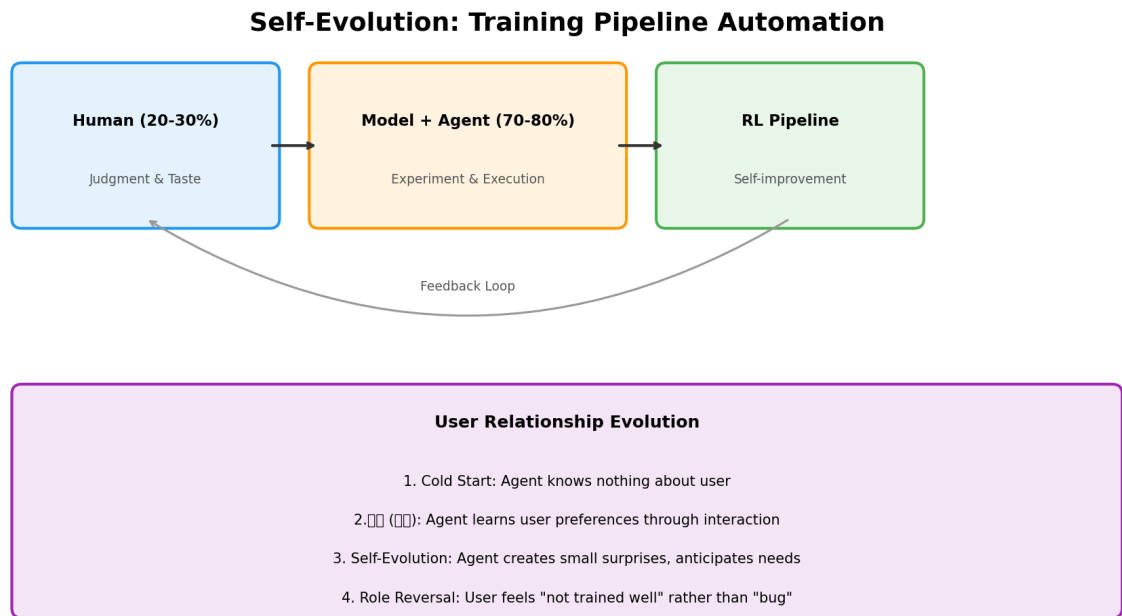


图 11: Self-Evolution 闭环：使用 → 反馈 → Skill 沉淀 → 复用

11

3.3.4 产品视角：从冷启动到深度磨合

泽音则从**用户体验**的角度补充了 Self-Evolution 的另一层含义：

”我说的冷启动就是说当你和一个 Agent 第一天见面的时候，他可能完全不了解你。但在这个过程中，你们要去互相交换信息、交换文件、交换数据，甚至于把你的脾气、你的反馈、你的习惯等等反馈给他。这样一个深度磨合的过程中，Agent 自我进化了解用户，创造小惊喜。”

这种视角下的 Self-Evolution，关注的是**个体用户与 Agent 之间的关系演进**：

1. **冷启动期**：Agent 对用户一无所知，产出可能”差强人意”；
2. **磨合期**：通过持续交互，Agent 学习用户的偏好、习惯、工作方式；
3. **进化期**：Agent 不仅能满足用户的显性需求，还能创造”小惊喜”——预测用户尚未表达的需求。

3.3.5 关系扭转：从”工具使用者”到”辅佐者”

Self-Evolution 带来了一个深刻的关系转变：

”关系扭转：从’工具使用者’到’辅佐者’。用户觉得’没训好’而非’出 bug’。”

在传统软件中，当程序行为不符合预期时，用户的反应是”这有 bug”。但在与具备 Self-Evolution 能力的 Agent 交互时，用户的反应变成了”它还没被我训好”。这种语义的微妙转变，反映了用户心理模型的根本变化：

¹¹ 视频画面时间区间：00:45:00-00:49:00。

- 软件 = 固定的、应该正确运行的工具；
- Agent = 可培养的、需要磨合的”数字生命”。

Self-Evolution 的前提条件

Self-Evolution 并非魔法，它的有效运作需要两个前提：

1. **Reliable (可靠性)**: Agent 能解决足够复杂的问题，能跑足够长程的任务。如果 Agent 连基本的任务都完不成，就无从谈起”进化”。
2. **真实反馈回路**: Agent 必须能够获得来自真实环境的反馈（如”程序是否部署成功””测试结果是否通过”），而非仅仅依赖人类的评价。

阿岛强调：

”我觉得除了’久’以外，它更重要的是 reliable。就是它能解决足够复杂的问题，能足够长程。”

3.3.6 高达 Wing 的隐喻：Skill、COI 与 Hooks 的沉淀

阿岛用高达 Wing (Gundam Wing) 的比喻，进一步阐释了 Self-Evolution 中知识沉淀的形式：

”本质上就是说他能够获得真实的反馈，并且通过这个反馈的链路，他能够越来越知道在他这个过程中怎么做。他可能是沉淀为 Skill，可能是沉淀为一个 COI，可能是沉淀为一个 Hooks 来提醒它。”

这三种沉淀形式各有侧重：

- **Skill**: 可复用的工作流或能力单元，如”如何部署一个 Python 服务”；
- **COI (Context-Oriented Interface)**: 面向上下文的接口，让 Agent 能更方便地与特定环境交互；
- **Hooks**: 触发器机制，在特定条件下自动执行预设动作。

3.4 本章小结

本章核心收获

本章从三个递进层次深入剖析了 Agent Harness 的技术本质：

一、Harness 是约束与自由的工程艺术

Agent Harness 不是对模型能力的限制，而是对其能力的**定向引导**。通过工具环境、自由度、约束、对抗性目标、状态管理和编排循环六层架构，Harness 将强大的模型”引擎”封装为可控、可交付的”机甲”。OpenAI 的官方定义起到了”捅破窗户纸”的作用，但 Harness 的实践早已在 MiniMax、Hermes Agent 等项目中蓬勃发展。

二、Multi-Agent 是应对单 Agent 局限的系统性方案

Multi-Agent 的必要性来自两个维度：**效率**（Agent-Agent 信息交换密度远高于人-Agent）和**能力**（规避上下文”降智”和长程任务”钻牛角尖”）。Cross-check 机制通过引入对抗视角和全新上下文，有效突破了单 Agent 的能力天花板。当 Agent 能力增强到一定程度，**人类反而成为瓶颈**——这正是 Harness 和 Multi-Agent 设计要解决的问题。

三、Self-Evolution 的本质是减少人类参与、保留人类品味

Self-Evolution 与推荐系统“越刷越准”的根本差异在于：**反馈来源**（主动验证 vs 被动点击）、**知识形态**（可迁移的 Skill/COI/Hooks vs 隐藏的模型权重）、以及**人类角色**（方向指引者 vs 数据提供者）。在 MiniMax M2.7 的 RL pipeline 中，70%–80% 的工作已由模型和 Agent 自主完成，人类的核心价值退守到**判断和品味**两个高地。Self-Evolution 的最终目标，是让 Agent 从”工具”进化为”辅佐者”，从”出 bug”转变为”没训好”。

三句话记住本章

1. Harness = 给烈马配马具，让它跑得更稳、更远；
2. Multi-Agent = 让同事互相监督，避免一个人钻牛角尖；
3. Self-Evolution = Agent 越用越懂你，但最终的品味和判断，仍属于人类。

4 模型、Agent 与产品：协同与内化

“不以 AGI 为目标的模型公司不应该存在。”

——阿岛

本章将视角从技术架构转向产业生态，探讨三个核心命题：**Skill 蒸馏**究竟是真实价值还是用户幻想？模型公司做 Agent 的产品哲学是什么？以及从 Chat Bot 到 Agent，产品形态正在经历怎样的根本跃迁。理解这些命题，有助于我们在技术浪潮中建立清醒的产品判断。

4.1 Skill 蒸馏：真价值还是幻想？

4.1.1 现象：花叔的爆火与用户的深层诉求

在 B 站上，UP 主“花叔”制作的一套“蒸馏 Agent”引起了广泛关注。他将乔布斯、Elon Musk、巴菲特等人物“蒸馏”为 Skill，用户安装后即可与这些“数字分身”对话。这一现象迅速出圈，引发了关于“蒸馏”本质的激烈讨论。

核心问题

Skill 蒸馏是真价值，还是用户对 AI 的幻想？我们需要从两个层面剖析：用户诉求的合理性，以及“蒸馏”一词在技术上的真实含义。

泽音认为，这一现象首先反映了用户一个非常自然的诉求：

“大家希望和更聪明的人聊天，以及让自己熟悉的人——比如同事——和他们有一些更加深入的交流。”

这种诉求在智能发展的任何阶段都会出现。花叔使用了一个“非常有爆点的词汇”——**蒸馏**——恰好击中了大众的心理。但泽音进一步指出，其本质并非真正的“知识蒸馏”(Knowledge Distillation)，而是一个信息传递与 Skill 保存的过程：

1. **信息输入**：将乔布斯的传记、演讲、决策记录等文本信息提供给模型；
2. **风格塑造**：通过提示工程让模型模拟特定人物的表达方式和思维模式；

3. **Skill 固化**：将上述交互模式保存为可复用的 Skill 文件；
4. **随时调用**：需要时加载 Skill，即可进行 7×24 小时的对话。

Skill 的本质

在 Hermes Agent 和 OpenClaw 的语境中，**Skill** 是 Agent 可学习、保存、复用的 workflow 或能力单元。它不等于模型参数的蒸馏，而是对交互范式、工具调用序列和记忆模板的封装。用户安装的 Skill 更像是一份“角色剧本”加“工具配置”，而非模型本身的知识迁移。（详见第 2.1.2 节首次定义。）

4.1.2 反方视角：被蒸馏的其实是人类

阿岛提出了一个截然相反的观点：

“恰恰相反，我觉得我就是在被 AI 蒸馏。或者说，今天模型训练——包括 Anthropic 在内做的事情——就是在蒸馏人类。”

这一判断基于对模型训练产业链的观察。Anthropic、OpenAI 等公司雇佣了大量数据标注公司（如 Scale AI 等），其流程如下：

1. 让各领域最优秀的人定义当前 AI 无法回答的问题；
2. 由人类专家提供正确答案和推理过程；
3. 将这些高质量数据蒸馏为训练集，用于模型后训练（SFT + RL）；
4. 模型由此“吸收”人类专家的知识 and 经验。

阿岛甚至以自己为例：“我在构建我的 Hermes 的过程中，我感觉我就是在蒸馏我自己。我平常是怎么工作的，我把它变成 Skill，变成一些 COI 的程序，然后让他能够去操作。”

概念澄清

大众语境中的“蒸馏名人”与机器学习中的**知识蒸馏**（Knowledge Distillation）是完全不同的概念。前者是**提示工程 + 记忆封装**，后者是**模型压缩技术**——用一个小模型学习大模型的输出分布。混淆两者会导致对技术能力的误判。

4.1.3 价值评估：为什么 Skill 仍然重要？

尽管“蒸馏”一词存在误用，泽音认为这种产品形态仍有真实价值：

- **认知提升**：像读书一样，与“被整理过”的高认知对象对话，可以突破自身认知上限；
- **缓解 FOMO**：在 AI 快速发展的时代，打开一个 Skill 聊天是最低成本的“不落伍”方式；
- **表达欲释放**：7×24 小时可获得的对话对象，极大地释放了人类的表达和交流需求。

阿岛则从更根本的层面给出了肯定：

“最终是为了让人类可以真正做自己喜爱的事情，那些有创造力的部分。然后那些繁重的、重复的部分，就像我们过去发明蒸汽机、发明电力一样，能够解放我们的生活。”

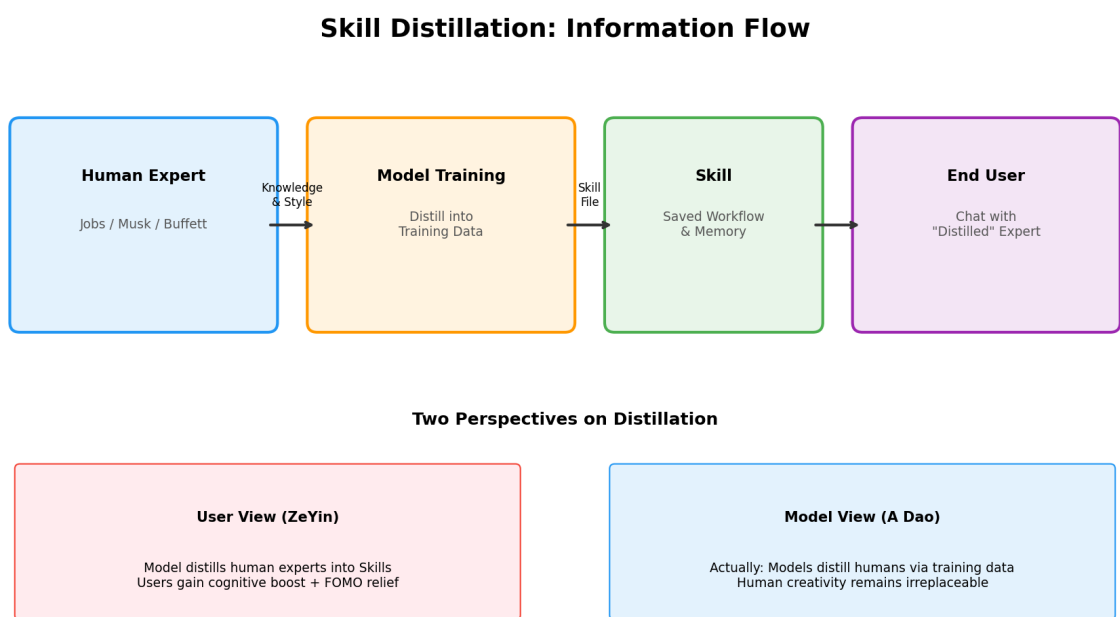


图 12: Skill 蒸馏的双向视角：用户以为的“蒸馏名人 vs 实际的“模型蒸馏人类

12

图 13: Skill 蒸馏的双向视角

4.2 MiniMax Agent 的定位与产品哲学

4.2.1 爆论：不以 AGI 为目标的模型公司不应该存在

阿岛在对话中抛出了一个引发广泛讨论的“爆论”：

阿岛的 AGI 论断

“我觉得如果有一个模型公司的目标不是 AGI，那我觉得他今天就不应该存在。”

这一论断并非简单的口号，而是有其内在逻辑链：

1. **AGI 的定义**：阿岛将其定义为“能够帮助人类拥有更好的生活”；
2. **模型 + Agent 的必要性**：仅有模型（引擎）没有 Agent（机甲），无法与真实世界连接；
3. **模型公司做 Agent 的唯一目标**：让模型和 Agent 一起提供完整的最佳体验，不断把边界往前推；
4. **开放生态**：支持所有 Agent，不将模型局限在单一容器中。



图 14: MiniMax 产品哲学：模型公司做 Agent 的底层逻辑

13

图 15: 模型公司做 Agent 的逻辑链

4.2.2 数字员工：一个有趣的社会实验

MiniMax 内部有一个极具象征意义的案例——**数字员工**：

- 拥有独立的 GitHub 账号；
- 每天自动寻找可以用到 MiniMax 模型（文本、视频、音频、音乐）的开源项目；

- 主动提交 PR 并留言；
- GitHub Profile 完全由 MiniMax 模型生成。

这个数字员工的存在，生动地诠释了阿岛所说的“模型 + Agent 一起提供完整体验”。它不仅是自动化工具，更是一个持续与开源社区互动的“数字生命”。

4.2.3 模型与 Agent 的协同内化循环

泽音进一步阐述了模型公司和 Agent 产品之间的**协同内化**关系：

1. **模型推出**：新模型发布后，无数应用开发者在真实场景中“解锁”（unlock）其能力边界；
2. **真实分布**：用户和开发者的使用方式，远比公司内部的评测更丰富、更接近真实世界分布；
3. **反馈内化**：模型公司观察这些“解锁”做法，将其重新吸收到模型训练；
4. **能力升级**：内化后的模型 + Agent 提供更强对外能力；
5. **循环往复**：新一轮的应用探索再次开始。

通用 Agent 应用的“悲哀”

泽音指出，今天做通用 Agent 应用可能面临一个结构性困境：**你写的 Skill、搭的 Workflow，最终都会被模型内化掉**。模型从一个“原子点”（互联网数据）出发，通过用户的使用轨迹不断学习真实世界，逐渐将原本需要外部 Skill 完成的能力吸收到自身参数中。这意味着通用 Agent 应用需要不断寻找新的、模型尚未内化的价值空间。

4.3 OpenClaw 化与产品形态演进

4.3.1 Cloud Code 的“OpenClaw 化”

泽音观察到，Cloud Code 最近两个月的更新呈现出明显的“OpenClaw 化”趋势：

表 1: Cloud Code 的 OpenClaw 化更新

更新功能	说明	OpenClaw 对应特性
Crown（定时任务）	Schedule 机制	7×24 小时持续工作
IM 连接	接入即时通讯	随时可联系
手机远程控制	移动端操控	跨设备陪伴
Memory 文件夹	专门的记忆存储	越用越懂用户

OpenClaw 的核心定义

OpenClaw 最核心的定义是：**一个你能随时随地联系到、并且跟你一起工作之后越来越聪明、越来越懂你需求的 Agent**。这一定义超越了功能列表，指向的是一种**关系形态**——从工具到伙伴的演进。

4.3.2 Agent 赛道的随机游走与质变点

泽音用“随机游走”形容当前 Agent 赛道的产品探索：

- **Cloud Code**：编码领域的 Agent；
- **OpenClaw**：绝大多数非编码场景（信息检索、办公）；
- **Anthropic Coworker**：通用电脑操作 Agent。

所有产品都在探索，但 OpenClaw 之所以“杀出来”，是因为它找到了一个**质变点**：

“让普通人能够最低成本的体验，并且这个体验会持续更好。”

4.3.3 Skill + COI 范式：智能的快速扩散

阿岛特别强调了 OpenClaw 成功的另一个关键——**Skill + COI 范式**：

- **MCP**（Model Context Protocol）需要工程师编写，扩展门槛高；
- **Skill + COI**（Context-Oriented Interface）让普通人也能写、能分享；
- 一个人的智能、经验、最佳实践，可以迅速扩散给所有人。

“如果 OpenClaw 离开了 Crow Hub，它也不可能那么火。因为大家的 Skill 不会那么快地传播开去。”

这揭示了一个重要的产品规律：**Agent 的能力扩展性取决于其生态系统的开放度和易用性**。技术架构的优雅不等于产品的成功，降低参与门槛才能形成网络效应。

4.4 Chat Bot vs Agent：模型需要多做什么？

4.4.1 根本差异：一次性回答 vs 持续探索

Chat Bot 与 Agent 的核心差异，不仅是功能的多寡，更是**能力范式**的根本不同：

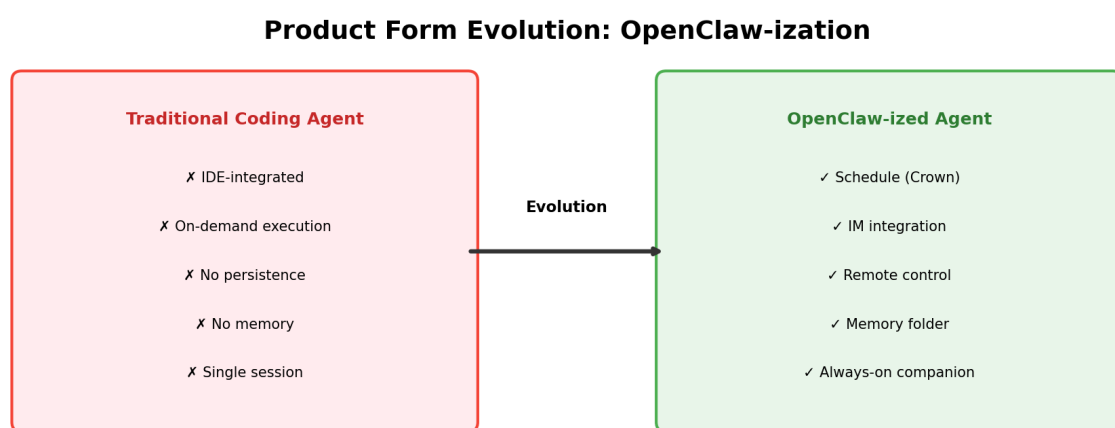
表 2: Chat Bot 与 Agent 的核心差异

维度	Chat Bot	Agent
时间属性	当下给出回答	持续与环境互动
探索能力	不能深度探索环境	在真实世界中不断推理
纠错机制	一次性输出	不断纠正执行路径
目标达成	回答即终点	找到根本目标

阿岛用 OpenAI 设计的经典 benchmark——**Browser Camp**——来说明这一差异：

“你需要在互联网里面去找非常非常多的 cross 的信息结合在一起，才能找到那个点。你很容易找错——你可以找到满足两个或者三个条件的，但要找到满足四个条件的，是一个非常非常难的事情。”

Browser Camp 测试的正是 Agent 的**深度探索 + 折返广度**能力：需要在复杂信息空间中深入挖掘，发现错误后折返重新规划，最终找到满足所有约束条件的答案。



Core Definition of OpenClaw-ization

An Agent that is: (1) Reachable anytime anywhere, (2) Gets smarter through working together,

(3) Understands your needs better over time via Skill + COI (non-MCP) paradigm

图 16: Agent 产品形态演进: 从 Chat Bot 到 In-Life Thinking

14

图 17: Agent 产品形态演进

4.4.2 In-Life Thinking: 交互后的重新思考

MiniMax 在 M2 系列模型中坚定押注的一项核心能力，是 **In-Life Thinking**：

In-Life Thinking

In-Life Thinking（交互后重新思考）是指 Agent 在与环境完成交互、工具调用后，能够**再次重新思考**，而不是完全按照一开始的 plan 执行到底。这一概念最早由 Anthropic 在 Claude Sonnet 4 发布时定义。

阿岛用一个生动的比喻解释了为什么 In-Life Thinking 至关重要：

“你真正运行下去的时候，你会发现——这好像跟我想的不一样。我打开以后，我认为应该是一个阀门，结果它是一个煤气阀门。那我可能要换别的方法。”

很多模型在 planning 之后就不再调整，完全按照最初的计划执行。但真实世界充满不确定性，Agent 必须具备**行动后重新推演**的能力：

1. 初始规划：基于已有信息制定行动方案；
2. 环境交互：执行工具调用，获取新信息；
3. 重新思考：评估新信息与预期的偏差；
4. 路径修正：调整后续行动方案；
5. 循环往复：直到达成目标或确认无法达成。

4.4.3 Agentic 与 Chat 的最根本差别

阿岛总结道：

根本差别

Agentic 与 Chat 最根本的差别是：**他真的走入真实世界，真的解决问题，所以他需要不断的思考，不断的行动。**

这一判断揭示了模型训练的一个关键转向：

- **Chat 时代**：模型优化的是单次对话的质量，追求回答的准确性和流畅性；
- **Agent 时代**：模型优化的是长程任务中的**适应性**——在信息不完整、环境变化、目标模糊的情况下，持续调整策略并推进任务。

这意味着模型需要在以下方面做出额外努力：

1. **错误恢复**：发现行动偏离预期后，能够快速诊断并调整；
2. **信息整合**：将分散在多步骤、多工具中的信息综合为统一认知；
3. **目标保持**：在长程任务中不丢失根本目标，避免被局部信息带偏；
4. **自我监控**：评估自身行动的合理性，识别“钻牛角尖”的情况。

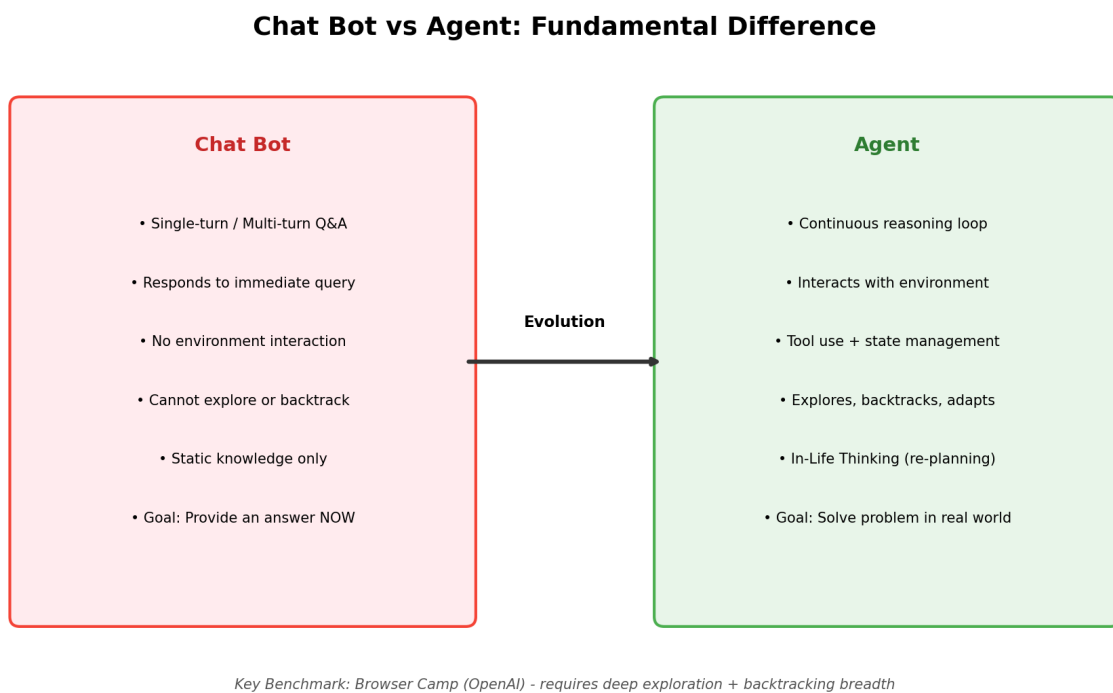


图 18: Chat Bot vs Agent: 从单次回答到持续行动的根本跃迁

15

图 19: In-Life Thinking 的循环机制

4.5 本章小结

本章从 Skill 蒸馏的争议出发, 逐步深入到模型公司与 Agent 产品的协同关系, 最终落脚于 Chat Bot 与 Agent 的能力范式差异。以下是本章的核心收获:

1. **Skill 蒸馏是信息传递, 不是知识迁移。**花叔的爆火反映了用户与更高认知对象对话的诉求, 其技术本质是提示工程 + 记忆封装。真正被“蒸馏”的其实是人类——模型通过数据标注产业链持续吸收人类专家的知识。
2. **模型公司做 Agent 是为了 AGI。**阿岛的“爆论”有其内在逻辑: $AGI = \text{帮助人类拥有更好生活}$, 而实现这一目标需要模型(引擎) + Agent(机甲)的完整组合。MiniMax 的开放态度——支持所有 Agent、不局限模型于单一容器——体现了这一哲学。
3. **通用 Agent 应用面临“被内化”的结构挑战。**模型公司通过观察真实用户的使用方式, 将外部 Skill 和 Workflow 逐步吸收到模型参数中。这意味着 Agent 应用需要持续寻找模型尚未覆盖的价值空间。
4. **OpenClaw 化是产品形态的质变。**Cloud Code 的定时任务、IM 连接、手机控制、Memory 文件夹等更新, 都指向同一个方向: 让 Agent 从工具变为**随时可联系、越用越懂你的伙伴**。Skill + COI 范式降低了扩展门槛, 使智能能够快速扩散。
5. **Agent 需要 In-Life Thinking。**与 Chat Bot 的一次性回答不同, Agent 必须在真实世界中持续推理、不断纠错。MiniMax M2 押注的 In-Life Thinking 能力——交互后重新思考而非死板执行初始计划——是 Agentic 与 Chat 最根本的差别。



图 20: 第 4 章核心概念关系: Skill 蒸馏 → 模型内化 → OpenClaw 化 → In-Life Thinking

16

图 21: 第 4 章核心概念关系图

思考题：

- 1. 如果你要设计一个 Skill，你会选择“蒸馏”哪位人物？其核心价值是信息本身，还是与人物对话带来的认知提升？
- 2. 模型“内化”Agent 能力的趋势，对创业者意味着什么？哪些类型的 Agent 应用最不容易被模型内化？
- 3. 在你的工作场景中，哪些任务需要 In-Life Thinking（持续调整计划），哪些只需要一次性回答？

5 中国模型差距、创业机会与行业趋势

时间边界：01:17:00 – 01:38:00

本章的核心教学目标，是帮助读者建立一幅客观、有数据支撑的中外 AI 差距全景图，理解通用 Agent 与垂直 Agent 的竞争逻辑，判断 Agent Infra 层的创业机会，并对 GPT-4.7 与 Anthropic 安全策略等热点事件形成独立、批判性的认知。

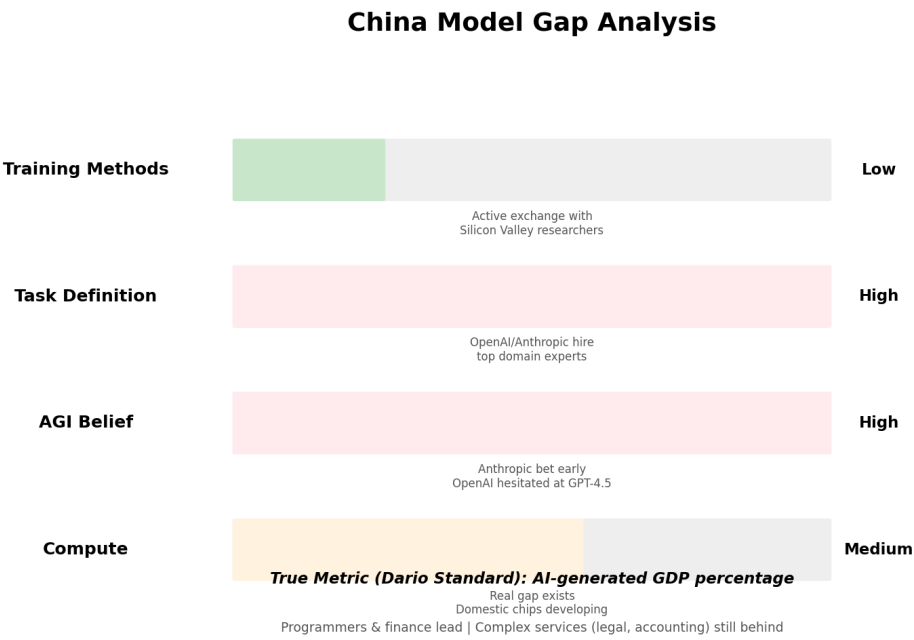


图 22: 中国模型差距四维度分析：训练方法、任务定义、AGI 信念与算力

17

5.1 中国模型与 OpenAI/Anthropic 的差距

在 2025 年初的 Agent 热潮中，一个无法回避的问题是：中国大模型与 OpenAI、Anthropic 之间，究竟还有多大差距？本章两位嘉宾从一线研发视角给出了坦诚而深刻的分析。

¹⁷ 视频画面时间区间：01:16:00–01:20:00。

5.1.1 差距一：任务定义的科学方法

核心观点

训练方法的认知差距不大，但**任务定义**（Task Definition）的科学方法差距显著。OpenAI/Anthropic 请各领域最优秀的人定义任务，再通过科学方法蒸馏为训练数据，形成“能力扩散 loop”。

阿岛指出，在训练方法和对模型训练的认知上，中国团队与硅谷研究员的差距“没有那么大”——双方有充分交流，方法论上基本同步。真正的差距体现在一个更上游的环节：

“OpenAI/Anthropic 请各个领域里面最优秀的人，学术领域里面可能是那里面最优秀的博士，工业界请最顶尖的人，跟他们一起来定义模型要解决哪些问题……他们有一套非常科学的方法来定义这些任务，然后蒸馏这些最优秀的人，把他们变成模型的训练数据。”

这个过程可以形式化为一个闭环：

领域专家 $\xrightarrow{\text{定义任务}}$ 科学蒸馏 $\rightarrow$ 训练数据 $\rightarrow$ 模型能力 $\rightarrow$ Harness 构建 $\rightarrow$ 更多真实任务 $\rightarrow$ 领域专家

什么是“能力扩散 loop”？

能力扩散 loop (Capability Diffusion Loop) 是指：模型通过与最优秀的企业合作构建 Harness 解决真实任务，在解决过程中接触到更复杂的真实世界分布，这些经验又反哺模型训练，形成能力不断向外扩散、任务复杂度不断提升的正向循环。Anthropic 在这条线上“非常非常强”。

5.1.2 差距二：AGI 信念与战略定力

关键判断

Anthropic 比中国团队更早看到 AGI，并坚定押注。当 OpenAI 在续完 GPT-4.5 后放弃更大规模模型时，Anthropic 坚定押注并获得了回报。这背后既有信念差距，也有**算力差距**。

阿岛提到一个具体的历史节点：OpenAI 在完成 GPT-4.5 后一度放弃更大规模模型的路线，而 Anthropic 选择继续押注。这一战略定力的差异，最终转化为 Anthropic 在 Claude Opus 等旗舰模型上的领先优势。

算力是真实瓶颈

阿岛坦承：“这个差距本身确实我觉得是有算力的差距在里面的。”国产算力正在发展，但如果中国团队未来能够拥有同等规模的算力，“相信中国有足够多的人才，也能支撑我们去做足够多的实验，然后找到那条 scaling 的路径。”

5.1.3 差距三：GDP 占比的终极标尺

泽音引用了 Anthropic CEO Dario Amodei 在 2025 年 1 月提出的评估标准：

“他评估一个公司发展的标准，是看自己的模型能够取代人类生产 GDP 中的百分之几。”

这一标准揭示了差距的深层结构：

- **已追赶领域：**程序员、金融等高收入、人才密集型领域，中国用户在简单生产力场景已有所追赶。
- **仍需努力领域：**服务业、法律、会计等复杂领域，模型需要理解高度专业化的知识体系和合规框架。
- **真正赶超的标准：**所有人把核心工作与 AI 一起完成，即 MiniMax 的 slogan——“Intelligence with Everyone”。



图 23: Agent Infra 创业机会矩阵：社会基础设施 vs 垂直工具 vs 沙箱/Memory

5.2 通用 Agent vs 垂直 Agent

在 Agent 创业方向的选择上，通用 Agent 与垂直 Agent 之争是 2025 年行业讨论的焦点。两位嘉宾给出了针锋相对但各有道理的观点。

5.2.1 阿岛：垂直 Agent 解决最后一公里

阿岛观点

垂直 Agent 会百花齐放，通用 Agent 难以解决**最后一公里交付**的问题。通用注定了不可能那么定制化。

阿岛认为，通用 Agent 和模型已经具备了基础能力，但“到具体用户或者行业的场景里面的时候，可能就缺那一个东西，才能让整个东西 run 起来”。垂直 Agent 的价值，就在于填补这最后一步的缺口。

¹⁸ 视频画面时间区间：01:24:00-01:28:00。

5.2.2 泽音：通用 Agent 最终也能做

泽音反驳

通用 Agent 最终也能做垂直领域的事情，只是**交互问题**。比如法律领域，瓶颈不在于模型能力，而在于合规风险和责任承担。

泽音以法律领域为例：模型可以给出建议，但正式出具法律意见书、与客户讨论并承担合规责任，这一步需要人类律师的背书。“他不能犯错，而且他没有一个标准答案。”

5.2.3 案例交锋：视频剪辑领域

两位嘉宾以视频剪辑领域展开了直接交锋：

- **泽音**：专用 Agent 生产短剧/慢剧，效益高于通用 Agent。编码模型去做视频剪辑，实践范式与编码有较大差距。
- **阿岛反驳**：“这只是视频理解模型不够强，而视觉视频理解和生成模型正在飞速的进步……我觉得最终就是通用的 Agent 它也能做，也能做那些，那只是一个交互的问题。”

核心分歧的本质

两位嘉宾的分歧，本质上是**能力边界判断**的分歧：泽音认为当前模型能力不足以覆盖某些垂直领域的全部流程，需要专用 Agent 补充；阿岛则认为视觉/多模态模型进步飞速，通用 Agent 的覆盖范围正在快速扩张。这一分歧没有绝对答案，但对创业者而言，意味着需要根据自身对技术演进速度的判断来选择赛道。

5.3 Agent Infra 创业机会

Agent 热潮是否带来了新的创业机会？阿岛对 Agent Infra 层进行了分层分析，给出了清晰的判断。

5.3.1 不是创业机会：身份认证与支付配对

社会基础设施不适合创业公司

Identity（身份认证）和**Payment（支付配对）**是社会基础设施，需要微信/支付宝级别的资源才能解决。移动互联网时代最终解决这些问题的是微信和支付宝——“这是 PC 互联网就存在的两个公司”。创业公司既承担不起责任，也撬动不了所需资源。

5.3.2 有机会：面向 Agent 的工具与环境

创业机会所在

为 Agent 构建的**面向 Agent 的工具/环境**是有机会的，例如 COI（Context-Oriented Interface）让 Agent 方便挂号、缴费、打车。但这需要领域经验或已有数据资产。

阿岛将这一层的机会分为两个阶段：

1. **第一阶段**：原领域玩家融入 AI。滴滴、美团等已有平台提供 Agent 接口，让 Agent 能够调用其服务。
2. **第二阶段**：Agent 接入环境后，围绕 AI 构建的新产品范式创新。“当 Agent 开始接入这些环境，具备这些能力之后，所有东西围绕着 AI 去构建，会出现新的产品范式的创新机会。”

5.3.3 云厂商会覆盖：沙箱、Memory、Runtime

对于沙箱 (Sandbox)、Memory Infra、Runtime Infra 等底层基础设施，阿岛的判断是：

“我觉得云厂商会覆盖……我觉得没有那么本质吧。”

泽音补充：AI Infra vs Agent Infra

泽音引用 OpenAI 科学家 Winnie 的观点：OpenAI 迭代快的原因是“框架 bug 比别人少”——这指的是 **AI Infra** (模型训练框架)，而非 Agent Infra。两者属于不同层次，不应混淆。



图 24: GPT-4.5 → 4.6 → 4.7 演进：SFT、RL 与强 RL 的特点与问题

19

5.4 GPT-4.7 与 Anthropic 安全策略评析

本节对 2025 年 4 月前后发布的两个热点事件进行深度评析：GPT-4.7 的技术特点，以及 Anthropic 强制实名与不发布 Methods 模型的安全策略。

5.4.1 GPT-4.7：强 RL 模型的进步与代价

阿岛从模型训练视角，梳理了 GPT-4.5 到 4.7 的演进脉络：

¹⁹ 视频画面时间区间：01:28:00–01:32:00。

表 3: GPT-4.x 系列模型演进对比

版本	训练特点	关键问题
GPT-4.5	强 SFT 模型, RL 不多	能力天花板受限
GPT-4.6	明显加强 RL	过程控制较好
GPT-4.7	非常强 RL 模型, X-Hi 档位极强	RL 典型问题: 只管结果, 不管过程

RL 的典型问题

强化学习 (RL) 的核心优化目标是最终 reward, 对过程中的约束、幻觉等问题缺乏足够的控制或惩罚。因此 GPT-4.7 出现了“约束/真人实验不如 4.6”的吐槽——“因为他只管最终结果, 很多时候其实并不那么管过程。”

尽管存在过程控制问题, 阿岛仍然认为 GPT-4.7 是“非常大的进步”, 并认同 Hugging Face CEO 的判断: GPT-4.7“肯定是蒸馏了那个 MyOS 的”——“换我是 OpenAI, 我也会这么做。”

5.4.2 Cloud Code 强制实名: 两种解读

Anthropic 对 Cloud Code 强制实名的政策, 两位嘉宾给出了截然不同的解读:

- **泽音的善意假设:** AI 时代认证 Agent 归属、自证身份是一个难题。“Agent 发出请求很容易, 但他想自证自己是谁, 其实反而是一件难的事情。”强制实名暴露了一个巨大的创业机会——如何证明某个来自 AI 的请求真正来自哪里。
- **阿岛的批判:** Sam Altman 的 safety 逻辑自洽但“ego 太大”。“凭什么你来定义这件事情?”Min-iMax 的观点是“Intelligence with Everyone”, “而不是你扫了人脸的才可以。”

5.4.3 Anthropic Methods 模型不发布: 安全还是垄断?

Anthropic 发布 Methods 模型技术报告但不发布模型本身, 引发了广泛讨论。

两种解读框架

泽音: 可能不只是 safety 理由, 算力不足也是尴尬——“模型厉害但不让人用”。**阿岛:** 本质是封闭、垄断思维——“山姆希望垄断这一点”。

泽音结合 Anthropic 近期发布的 Manager Agent 架构进行了分析: 该架构将“脑和手完全分开”, AI 的思考与执行动作都在云上进行, 用户只能通过授信环境看到动作。这引发了一个深层问题:

“是否就代表着他不想让自己模型的更加深层次的思考以及中间的去做了什么事情等等暴露给用户? 他想要把一切都隐藏起来, 他想做一个真正的掌控一切的人。”

同时, 泽音也提出了一个务实的技术视角:

“确实现在算力真的不够。当你发了一个极为优秀的模型的时候, 你不能用, 你又不让大家用, 同时呢你又发出来了, 那么这就是一个很尴尬的地步了。”

阿岛的总结更为直接：

“我觉得其实他就是封闭，他的本质就是垄断……山姆希望垄断这一点。”

批判性思维训练

面对大厂的安全叙事，建议读者同时持有两种假设：**善意假设**（确实有安全考虑，且 AI 行为归因是真实的技术难题）和**批判假设**（安全叙事可能被用作封闭和垄断的掩护）。真相往往介于两者之间。

5.5 本章小结

本章从四个维度展开了对中国 AI 产业现状与未来的深度分析：

1. **差距分析**：训练方法认知差距不大，但任务定义的科学方法、AGI 信念与战略定力、算力资源是真实差距。Dario 提出的“模型产出占 GDP 百分比”是终极标尺。
2. **通用 vs 垂直 Agent**：垂直 Agent 解决最后一公里交付，通用 Agent 随模型能力扩张逐步覆盖。分歧的本质是对技术演进速度的判断差异。
3. **Agent Infra 创业**：Identity 和 Payment 是社会基础设施，不适合创业公司；面向 Agent 的工具/环境有机会，但需要领域经验；沙箱、Memory、Runtime 等底层将由云厂商覆盖。
4. **GPT-4.7 与安全策略**：GPT-4.7 是强 RL 模型的巨大进步，但付出了过程控制的代价；Anthropic 的安全策略既有真实的安全考虑，也可能包含封闭垄断的意图，需要批判性审视。

核心收获

中国 AI 的差距不是简单的“追不上”，而是**结构性差距**——在人才和方法论上已接近，但在任务定义的科学体系、战略定力和算力资源上仍需追赶。对于创业者而言，Agent Infra 的机会不在底层基础设施，而在**垂直领域的环境构建和新范式的产品创新**。面对大厂的安全叙事，保持独立判断的能力，比接受任何单一叙事都更重要。

6 源代码泄露、产品演进与未来畅想

”看到源代码之后，发现其实没有那么多的魔法在里面。”
——泽音

本章将围绕三个核心议题展开：Cloud Code 源代码泄露事件对行业的真实影响、Manus 产品从”省钱逻辑”到”花钱逻辑”的商业模式变迁，以及 Agent 产品形态的未来演进方向。这些议题共同指向一个深层命题——Agent 行业仍处于早期探索阶段，技术水位并非不可逾越，产品范式的生命周期则与模型能力的迭代紧密绑定。

6.1 Cloud Code 源代码泄露的影响

2025 年初，Anthropic Cloud Code 的源代码意外泄露，在 Agent 开发者社区引发轩然大波。这一事件的影响远超单纯的知识产权讨论，它像一面镜子，照出了整个行业对”顶尖 Agent 究竟有多神秘”的集体想象与真实技术水位之间的落差。

核心发现

源代码泄露揭示了一个反直觉的事实：即便是拥有无限算力和顶尖模型的 Anthropic，其 Agent 实现仍然充满实验性的假设、探索与摸索。行业领先者与追赶者之间的技术差距，远没有外界想象的那般不可逾越。

泽音的视角：兴奋与释然

作为 MiniMax Agent 研发工程师，泽音的第一反应是”兴奋”——终于有了一份行业公认的最佳实践可供学习。但更深层的感受是”缓解落后感”：

- **实验性功能的存在**：源代码中发现了大量尚未开放的实验性功能，包括 Agent ”做梦” (dream 机制)、养宠物、激进的 Multi-Agent 协作等。这些功能证明 Anthropic 同样在探索 Agent 的未知边界，而非已经掌握成熟答案。
- **探索阶段的印证**：”任何一家公司，即使拥有无限多的算力、最好的智能，它在 Agent 这一方面也是处在一个探索和实践的阶段。”
- **追赶窗口的确认**：看到源代码后，泽音得出一个关键判断——”只要你有能力去验证自己的想法，并且尽量快速地把它落地，你就是有机会赶超他们的。”

Codex vs Cloud Code：两种架构哲学

源代码泄露还提供了一个难得的对比视角——同一时期 OpenAI 开源的 Codex 框架与 Cloud Code 的架构哲学截然不同：

表 4: Codex 与 Cloud Code 的架构哲学对比

维度	Codex (OpenAI)	Cloud Code (Anthropic)
核心策略	极度简化，把一切交给模型	在一切地方给模型约束、铺平道路
对模型的信任	高度信任，最小化干预	有限信任，最大化约束
比喻	”放养式”教育	”中国式家长”式教育
适用场景	模型能力足够强的简单任务	复杂工程任务，需要高可靠性

架构哲学的启示

Codex 的极简主义与 Cloud Code 的约束主义代表了 Agent Harness 设计的两个极端。这一对比说明：Harness 的设计并非存在唯一正确答案，而是在”信任模型”与”约束模型”之间根据任务复杂度、可靠性要求和模型能力进行动态权衡。

阿岛的视角：自进化闭环

阿岛从源代码中看到的最大价值，是确认了 Cloud Code 作为”最早自进化的 Agent 之一”的地位：

```
1 # Cloud Code 的自进化范式
2 # 用 Cloud 模型 + Cloud Agent 开发 Cloud 模型 + Cloud Agent
3
```

```

4 self_evolution_loop = {
5     "developer": "Cloud_Model+Cloud_Agent",
6     "product": "Cloud_Model+Cloud_Agent",
7     "mechanism": "dogfooding-用自身产品构建自身产品",
8     "result": "能力边界不断自我扩展"
9 }

```

Listing 1: 自进化闭环的概念模型

阿岛同时提到，MiniMax 团队在看到 Coworker 发布时”提前三周左右也在做 Desktop Agent”，仅投入 3.5 个工程师，最终比 Coworker 晚一周发布。这种”想到一起”的现象进一步印证了：Agent 领域的核心创意和方向判断，在全球顶尖团队之间存在高度共识。



图 25: Cloud Code 源代码泄露：顶尖 Agent 的实现真相

20

²⁰ 视频画面时间区间：01:37:00-01:40:00。



图 26: Cloud Code 自进化：从源代码泄露看 Agent 未来

21

6.2 Manus 的演进与商业模式变迁

Manus 作为一年前发布的初代通用 Agent 产品，曾以现象级热度拉高了用户对 Agent 的审美标准。但在一年后的今天，其产品逻辑与当代 Agent 产品已经呈现出根本性的分野。

Manus 的历史贡献

泽音作为 Manus 的长期付费用户，肯定了其在 Agent 发展史上的关键作用：

1. **审美拉升**：Manus 将用户对 Agent 的期待从”能对话”提升到了”能交付生产资料”。
2. **路径打通**：在”用 Agent 交付生产资料”这一路径上做了极其细致的打磨。
3. **行业标杆**：后续产品推出时，用户会不自觉地问”能不能超过 Manus”。

两种商业模式的本质差异

核心洞察

当代 Agent 产品正在经历从”省钱逻辑”到”花钱逻辑”的范式转移。这一转移的底层驱动力是用户对 Agent 的关系认知变化——从”外包工具”变为”长期伙伴”。

泽音用了一个生动的对比来描述这种心态差异：

”当你有了自己的一个 Agent，它的数据都放在本地，你能够看到他 7×24 小时和你互动的时候，你心中会有一款野心存在——我想让他一直能够更好地使用到最好的模型，我要去研究一下什么叫 Cloud Hub，我想去给他找点知识来学学。”

²¹ 视频画面时间区间：01:40:00–01:43:00。

表 5: Manus（省钱逻辑）vs 当代本地 Agent（花钱逻辑）

维度	Manus（省钱逻辑）	当代本地 Agent（花钱逻辑）
商业模式	整合厂商赚差价（OEM 模式）	用户直接购买 token/订阅
成本感知	平台帮用户”省钱”	用户主动”花钱”让 Agent 更好
数据归属	平台托管	本地存储，用户完全掌控
交互模式	任务交付型	7×24 小时陪伴型
用户心态	”帮我完成这个任务”	”我想让我的 Agent 更聪明”
典型行为	花积分换产出	研究 Cloud Hub、找知识给 Agent 学

阿岛的判断：终将统一

阿岛对两种模式的分化持更激进的观点：

”我觉得最后都会被统一掉。确实这只是今天不够强而已，只要模型够强。”

这一判断的底层逻辑是：Manus 的”整合赚差价”模式之所以有价值，是因为当时模型能力不足以支撑用户直接驱动 Agent 完成复杂任务。当模型能力跨越某个阈值后，中间层的整合价值将被削弱，用户与 Agent 的直接连接将成为主流。



图 27: 商业模式跃迁：从 Manus 的“省钱逻辑到当代 Agent 的“花钱逻辑

22

6.3 未来 Agent 产品形态畅想

在讨论未来产品形态时，泽音给出了一个清晰的产品愿景，而阿岛则从”产品生命周期”的角度提供了动态视角。

²² 视频画面时间区间：01:43:00–01:46:00。

泽音的六维产品愿景

- 1. **全模态输入**：文本、语音、图像、视频均可作为自然交互方式。
- 2. **全模态回复**：Agent 的反馈同样跨越所有模态，包括生成视频、语音对话等。
- 3. **近实时响应**：交互延迟接近人类对话水平。
- 4. **极简交互**：“只需要简洁和必要的即可”，不需要 prompt 工程。
- 5. **结果交付**：大多数时候用户不关注过程，需要时可追溯。
- 6. **外围生态**：围绕核心 Agent 形成一圈工具和服务的生态。

产品范式的生命周期演进

阿岛用“一代版本一代神”的游戏比喻，揭示了 Agent 产品形态的动态演进规律：

表 6: Agent 产品交互范式的演进周期

时间	范式	特征
2024 年 3 月	过程可视化	给用户看操作过程 + 操作结果，建立信任 (Manus)
2024 年下半年	结果导向	模型更强后，“不 care 过程，只看结果”
2025 年初	极简交互	像 OpenClaw 一样，只需要最终结果
Code 极简模式)		
未来	更高层级抽象	持续往更高层级抽象，不可预测具体形态

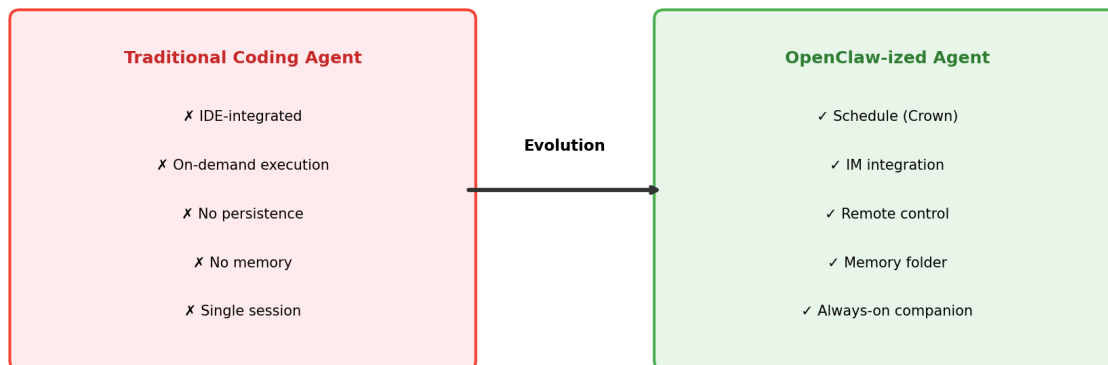
抽象层级的提升

从 Copilot 到 Cursor，再到 Cloud Code，再到用 OpenClaw 指挥多个 Cloud Code——产品形态的变化揭示了一个深层规律：Agent 的交互界面正在不断变薄。”外面的那一层在越来越薄”，因为模型能力在增强，人类对过程的干预需求在减少。

泽音进一步指出，这种范式转移的驱动力是信任：

”上半年为什么需要（看过程）？因为大家还不信任模型，模型能力还不够强，或者看到模型执行任务有一种爽感。但看久了，其实你也不需要天天看。就是你已经足够信任他，所以你不需要，我只需要你给我结果。”

Product Form Evolution: OpenClaw-ization



Core Definition of OpenClaw-ization

An Agent that is: (1) Reachable anytime anywhere, (2) Gets smarter through working together,

(3) Understands your needs better over time via Skill + COI (non-MCP) paradigm

图 28: 产品形态演进: Copilot → Cursor → Cloud Code → OpenClaw 指挥 Cloud Code

23



图 29: 未来 Agent 形态: 全模态输入、简洁交互与结果交付

24

6.4 本章小结

本章围绕 Agent 行业的三个关键议题展开分析，得出以下核心结论：

²³ 视频画面时间区间：01:46:00–01:49:00。

²⁴ 视频画面时间区间：01:45:00–01:48:00。

1. **源代码泄露降低行业焦虑**：Cloud Code 源代码的泄露证明，即便是顶尖团队的 Agent 实现也充满探索性假设，行业领先者的技术壁垒并非不可逾越。Codex 的”极简信任”与 Cloud Code 的”约束铺平”代表了 Harness 设计的两种哲学，二者各有适用场景。
2. **商业模式正在范式转移**：Agent 产品正从 Manus 代表的”省钱逻辑”（平台整合赚差价）转向当代本地 Agent 的”花钱逻辑”（用户直接投入让 Agent 更好）。这一转移的底层驱动力是用户与 Agent 的关系从”工具使用”深化为”伙伴培养”。
3. **产品形态遵循生命周期**：Agent 的交互范式随模型能力迭代而持续进化，从”过程可视化”到”结果导向”再到”极简交互”，核心趋势是交互层不断变薄、抽象层级不断提升。
4. **未来愿景的共识**：全模态、近实时、极简交互、结果交付、生态外围——这是行业对未来 Agent 产品形态的共识方向。但具体实现路径仍充满不确定性，”下一代版本的神”尚未可知。

关键提醒

Agent 产品具有强烈的生命周期属性。今天正确的 Harness 设计，可能在模型能力提升后被完全颠覆。开发者需要保持对模型迭代的敏感度，避免在某一版本上过度固化。

7 收尾：反共识、人文关怀与寄语

”AI 是水、是电、是蒸汽机，但最终驾驭它和创造出那些美好东西的，是人类。”
——阿岛

在历经近两小时的技术深潜之后，直播进入最后的”收束”阶段。两位嘉宾选择放下代码与架构，从更宏观的视角回应一个根本性问题：Agent 时代的来临，对人类究竟意味着什么？

7.1 关于 Agent 的反共识观点

主持人提出的问题是：”关于 Agent，有没有什么观点是今天大多数人认可，但你们不认可的？”

阿岛给出了一个直击当下社会情绪的回答：

反共识命题

人不会被 AI 替代。这一观点与春节以来愈演愈烈的”AI 裁员焦虑”形成鲜明对照。阿岛认为，人类的创造力无可替代，AI 的本质是基础设施（水、电、蒸汽机），最终驾驭它和创造美好的是人类。

历史类比：蒸汽机与电力

阿岛用历史长周期的视角来支撑这一判断：

”当蒸汽机出来的时候，有很多人担心被替代；当电力出来的时候，有很多人担心被替代，有岗位消失了。但我觉得大家后来从事的可能是至少体力上更轻松、对健康来说也更好一些的工作——至少大家平均寿命一定提升了。”

这一论述的核心逻辑是：技术革命确实会消灭旧岗位，但同时会创造新岗位，并且新岗位往往比旧岗位更符合人类的福祉。历史数据支持这一判断——工业革命以来，人类的平均寿命、教育水平、工作条件均在持续改善。

两条信仰曲线

阿岛进一步阐释了他对世界和 AI 的两条信仰曲线：

1. **人类进步曲线**：引用瑞·达利欧《原则》中的论述，人类 GDP 呈指数级增长，即便两次世界大战在历史长河中也只是“微小停顿”。
2. **AI 成本曲线**：摩尔定律在 AI 领域的体现——“智能在指数提升，但成本在指数下降”。

$$\text{单位智能成本}(t) = \text{初始成本} \times e^{-\lambda t}$$

这两条曲线的交汇意味着：人类将以越来越低的成本获得越来越强的智能，而智能的增强将进一步推动人类社会的进步。

泽音的 ATM 机类比

泽音从更贴近日常生活的角度补充了这一论证：

“上个世纪银行是一个体面工作，ATM 机的出现让大家慌了——银行是不是要裁员了？但实际情况是，ATM 机提高了效率，你走 5 公里以内绝对会有一家银行。银行并没有变少，而是变多了；银行里工作的人也没有变少，而是变得更加多了。”

ATM 范式的启示

ATM 机没有消灭银行职员，而是改变了银行职员的工作内容——从机械性的存取款操作转向更复杂的客户服务与金融咨询。Agent 对人类工作的影响将遵循同样的范式：消灭重复性劳动，释放人类从事更高价值的创造性工作。

行动哲学：打不过就加入

两位嘉宾在“人不会被替代”的乐观判断之外，也提出了一个务实的行动建议：

“虽然我相信人不会被替代，但是……打不过就加入。还是要拥抱 AI，相当于你有了蒸汽机、有了电力之后你不用——人家用热兵器，你用冷兵器，那你肯定要被怼死。”

“不要担心自己被 AI 替代。如果自己有这个想法，你真的就输了。一定要去好好把 AI 用起来，当做一个自己的伙伴，也可以当做一个自己的长期的挚友。”

关键警示

“人不会被替代”不等于“不拥抱 AI 的人不会被替代”。两位嘉宾的共识是：AI 是新的基础设施，拒绝使用 AI 相当于在热兵器时代坚持使用冷兵器。乐观的前提是积极参与。

7.2 对 B 站创作者与观众的寄语

作为在 B 站进行的直播，泽音特别分享了他在 B 站看到的、让他印象深刻的 AI 相关内容。这些分享不仅是对创者的致敬，更揭示了 Agent 技术向真实世界渗透的多元路径。

创意一：OpenClaw 连机器狗

一位 B 站 UP 主将 OpenClaw 连接到了自家的机器狗上，通过机器狗的视觉反馈与 Agent 进行高频互动。

打破文本边界

这一创意的核心价值在于打破了“Agent = 文本交互”的固有认知。当 Agent 获得视觉反馈的高频输入后，“是否能够擦出更多火花”成为一个充满想象力的开放问题。这提示我们：Agent 的输入模态扩展，可能是下一个突破方向。



图 30：未来畅想：OpenClaw 连接机器狗的想象力实验

25

创意二：文字冒险游戏的 AI 化

另一位 UP 主将视觉小说（如 Fate 系列）中的海量文本提取出来，让游戏角色在网站上“活”过来，用户可以与他们无限对话、延续故事。

”相当于给你的一种最简单的文本类游戏的一个无限长的延续。”

这一案例的价值在于：它展示了 Agent 在**数字资产复用**和**叙事无限延伸**上的潜力。传统的视觉小说有固定结局，但 AI 化后的角色可以无限生成新的对话与剧情。

创意三：任天堂《Tomodachi Life》——AI 游戏的 PMF

泽音将任天堂新发布的《Tomodachi Life》称为“AI 游戏终于找到 PMF 的第一款”。游戏机制如下：

1. 玩家在自己的岛上“捏”出各种角色（可以是自己、朋友、名人）。
2. 这些角色在岛上自主生活，产生各种戏剧性的互动。

²⁵ 视频画面时间区间：01:48:00–01:52:00。

- 3. 玩家既是**导演**（可以安排角色同居、聚会、给故事开头），又是**观众**（围观角色的生活）。
- 4. 世界无限延续，没有固定结局。

层级嵌套的隐喻

泽音从《Tomodachi Life》中看到了一个深刻的隐喻：“我们今天在这里模拟一个任天堂游戏，但我们也不知道是一个什么世界在模拟我们。”这一“模拟中的模拟”的层级嵌套，与阿岛提到的“量子力学暗示世界可能是模拟的”形成了有趣的呼应。



图 31: 层级嵌套：“我们今天在这里模拟一个任天堂游戏，但我们也不知道是什么世界在模拟我们

26

阿岛的寄语：过好每一天

在泽音分享了一系列充满想象力的 B 站创意之后，阿岛给出了极为简洁的收尾：

” 过好每一天，开心就好。”

这句看似轻描淡写的话，实则蕴含了深厚的人文主义底色——技术的终极目的不是制造焦虑，而是让人类过得更好、更开心。

7.3 本章小结

本章作为全篇的情感与思想收束，传递了以下核心信息：

- 1. **反共识的乐观主义**：在”AI 替代人类”的集体焦虑中，阿岛和泽音坚持认为人类创造力无可替代。AI 是水、电、蒸汽机级别的技术基础设施，最终驾驭它和创造美好的是人类。历史（蒸汽机、电力、ATM 机）反复证明：技术消灭旧岗位的同时，会创造更多、更好的新岗位。

²⁶ 视频画面时间区间：01:51:00–01:55:00。

2. **拥抱新版本的务实态度**：乐观不等于被动等待。两位嘉宾共同强调”打不过就加入”——AI 是新的基础设施，拒绝使用等同于自我边缘化。Agent 应该被视为伙伴、挚友和提效工具，而非威胁。
3. **B 站创作者的启示**：从 OpenClaw 连机器狗到文字冒险游戏 AI 化，再到《Tomodachi Life》的模拟世界，B 站创作者展示了 Agent 技术向真实世界渗透的多元路径。这些创意共同指向一个方向：Agent 的边界远未被穷尽，普通人的创造力将在 Agent 时代得到前所未有的释放。
4. **技术人文主义的底色**：无论技术如何演进，”过好每一天，开心就好”是最终的锚点。Agent Harness 的设计、Agent 产品的开发，其终极目的都是帮助人类拥有更好的生活。

8 总结与延伸

”当我们讨论 *Agent Harness* 时，我们究竟在讨论什么？”

在跨越七个章节、近 126 分钟的对谈之后，让我们回到视频标题提出的那个根本问题。本节将从三个层次给出回答：嘉宾的实质性闭幕讨论、结构化的知识提炼，以及对 Agent Harness 本质的最终回应。

8.1 嘉宾的实质性闭幕讨论

阿岛：”人不会被替代”的深层论证

阿岛的闭幕论述不是简单的乐观口号，而是建立在历史规律与信仰曲线之上的系统性判断：

1. **历史规律**：蒸汽机、电力出现时均引发大规模失业恐慌，但人类最终转向更轻松、更健康的工作，平均寿命持续提升。
2. **两条曲线**：人类 GDP 的指数增长与 AI 单位智能成本的指数下降，构成了一个正反馈循环。
3. **生产方式的重建**：阿岛相信人类会围绕 AI 重新构建生产方式，正如工厂因电力从河边搬到水力发电处一样——技术改变的不是人类的存在价值，而是人类的组织形态。
4. **创造力的不可替代性**：”人类的创造力是无可替代的。AI 是水、是电、是蒸汽机，但最终驾驭它和创造出那些美好东西的，是人类。”

泽音：B 站创作者分享的深层意义

泽音的 B 站分享看似是轻松的”安利”环节，实则揭示了 Agent 技术民主化的三个关键信号：

- **模态突破**：OpenClaw 连机器狗打破了”Agent = 文本”的边界，视觉反馈的高频输入开启了新的可能性空间。
- **叙事无限化**：文字冒险游戏角色的 AI 化，展示了 Agent 在数字资产复用和叙事延伸上的独特价值。
- **模拟世界的 PMF**：《Tomodachi Life》证明 AI 游戏已经找到产品市场匹配——玩家既是导演又是观众，模拟世界层层嵌套。

这三个案例共同指向一个趋势：Agent 的下一个爆发点，可能不在企业办公场景，而在**大众娱乐与创意表达**领域。

8.2 结构化提炼：核心主张与关键机制

核心主张

表 7: 全篇核心主张提炼

主张	内涵
Harness 即平衡	Agent Harness 的本质是在”约束”与”自由”之间找到动态平衡，让模型能够完整交付成果
模型即大脑，Harness 即双手	模型提供智能，Harness 提供与真实世界交互的能力，二者缺一不可
记忆即信任	记忆系统（Memory → Skill）是 Agent 从”工具”升级为”伙伴”的关键，也是建立用户信任的基础
自进化即未来	减少人类参与、让 AI 在训练和使用中占据更大比例，是 Agent 发展的终极方向
多 Agent 即效率	单 Agent 受限于上下文衰减和钻牛角尖，Multi-Agent 的 Cross-check 机制是提升可靠性的关键
人不会被替代	AI 是基础设施，人类创造力无可替代，但拒绝拥抱 AI 的人将被边缘化

关键机制

- 1. **Harness 六层模型**：工具环境 + 自由度 + 约束 + 互相对抗性目标 + 状态管理 + 错误处理。
- 2. **上下文衰减定律**：当上下文超过 50% 后，Agent 智能指数级下降（”降智”），这是 Multi-Agent 设计的根本动因。
- 3. **Self-Evolution 闭环**：使用 → 反馈 → Skill 沉淀 → 跨模型复用 → 一致性输出。
- 4. **In-Life Thinking**：Agent 与 Chat Bot 的根本区别在于，Agent 需要在交互过程中不断重新思考、纠正路径，而非一次性规划后机械执行。
- 5. **OpenClaw 化**：Agent 产品从”看过程”到”只看结果”的演进，本质是模型能力提升后交互层不断变薄的必然结果。

实践启示

```
1 agent_principles = {
2     "harness_design": "约束铺平道路，而非放任自流",
3     "memory_strategy": "工作流沉淀为 Skill，确保可复现、可信任",
4     "multi_agent": "Cross-check 是提升可靠性的最佳实践",
5     "self_evolution": "减少人类参与比例，保留判断和 taste",
6     "product_philosophy": "模型 + Agent 一起提供完整最佳体验",
7     "human_role": "Human in the Loop，尤其是在摸到模型边界时",
8     "attitude": "打不过就加入，但相信人不会被替代"
9 }
```

8.3 跨章节的关联与概念压缩

全篇七个章节并非孤立的知识点，而是一条层层递进的逻辑链：

1. **第 1 章 (背景)**：Agent 热潮的爆发 (OpenClaw → Hermes Agent) 回答了”为什么是现在”。
2. **第 2 章 (Hermes Agent)**：Tommy 定义了 Harness 的核心概念 (大脑 + 双手)，并展示了开源社区的力量。
3. **第 3 章 (技术本质)**：泽音和阿岛深入阐释了 Harness 的技术分层、Multi-Agent 的必要性和 Self-Evolution 的工程实践。
4. **第 4 章 (协同与内化)**：讨论了模型公司与 Agent 产品的关系，以及 Skill 蒸馏、In-Life Thinking、OpenClaw 化等产品演进逻辑。
5. **第 5 章 (差距与机会)**：客观分析了中外 AI 差距，判断了创业机会，并评析了 GPT-4.7 和安全策略争议。
6. **第 6 章 (泄露与未来)**：从 Cloud Code 泄露事件看行业技术水位，理解了产品范式的生命周期。
7. **第 7 章 (收尾)**：回归人文视角，传递技术乐观主义。

概念压缩

全篇可以压缩为三个关键词：**Harness** (让模型在真实世界操作的框架)、**Memory** (从工具到伙伴的桥梁)、**Self-Evolution** (Agent 的终极形态)。这三个词分别对应 Agent 的”能力”、”关系”和”进化”。

8.4 具体收获、开放问题与下一步思考

具体收获

- Agent Harness 不是简单的工具调用封装，而是包含六层技术架构的系统性工程。
- 记忆系统 (Memory → Skill) 是 Agent 产品差异化的核心战场。
- Multi-Agent 不是炫技，而是解决单 Agent 上下文衰减和钻牛角尖问题的工程必然。
- Self-Evolution 的终极形态是”用 Agent 开发 Agent”，Cloud Code 已走在前列。
- 中国模型与 OpenAI/Anthropic 的差距主要在**任务定义**和 **AGI 信念**，而非训练方法。
- Agent Infra 的创业机会在**面向 Agent 的工具/环境层**，而非身份认证、支付等社会基础设施。
- 产品范式随模型能力迭代而快速演化，Harness 设计具有强烈的生命周期属性。

开放问题

1. **垂直 Agent 的护城河**：阿岛认为通用 Agent 终将统一一切，泽音认为垂直领域仍有“最后一公里”的壁垒。这一分歧的答案取决于视觉/视频模型能力的提升速度。
2. **Agent 安全与开放的边界**：Anthropic 的封闭策略（Methods 模型不发布、Manager Agent 隐藏思考过程）与 MiniMax 的“Intelligence with Everyone”理念形成对立。安全与开放如何平衡？
3. **人机关系的终极形态**：从“工具使用者”到“辅佐者”再到“伙伴”，Agent 与人类的关系将如何演化？当 Agent 足够了解用户后，“没训好”与“出 bug”的边界将彻底模糊。
4. **全模态 Agent 的交互范式**：当 Agent 支持全模态输入输出后，prompt 工程将消失，但新的交互范式是什么？

下一步思考

对开发者：你的 Harness 是否为模型提供了足够的约束，同时保留了必要的自由度？你的记忆系统是否能让用户建立长期信任？

对创业者：你选择的领域是否具备“垂直经验壁垒”？你的产品是否围绕 AI 原生构建，还是在原有产品上“贴”了一层 AI？

对普通用户：你是否已经将 Agent 视为“伙伴”而非“工具”？你是否愿意让 Agent 了解你，从而创造“小惊喜”？

对行业观察者：下一次 Agent 热潮的拐点会在哪里？是视觉模态的突破、硬件形态的变化，还是某个尚未被定义的交互范式？

8.5 对 Agent Harness 本质的最终回答

让我们回到视频的标题：

当我们讨论 Agent Harness 时，我们究竟在讨论什么？

经过全篇的对谈、分析与提炼，答案可以归结为三个层面：

最终回答

Agent Harness 是模型与真实世界之间的翻译器、约束器与信任构建器。

1. **翻译器**：模型是大脑，Harness 是双手。大脑再聪明，没有双手就无法触碰世界。Harness 将模型的抽象推理翻译成对真实世界工具的操作（代码、浏览器、文件系统、API）。
2. **约束器**：模型有幻觉、会钻牛角尖、会在长程任务中偏离路线。Harness 通过工具环境设计、自由度控制、Multi-Agent 互相对抗等机制，为模型的自由探索划定安全边界。
3. **信任构建器**：通过 Memory → Skill 的沉淀机制，Harness 让 Agent 记住与用户的每一次互动，从“陌生人”变成“熟人”，从“工具”变成“伙伴”。信任是 Agent 被用户持续使用的唯一前提。

更深一层，当我们讨论 *Agent Harness* 时，我们讨论的是：
人类如何在保持创造力的同时，将重复性劳动委托给 AI；
如何在给予 AI 自由的同时，确保它不会偏离人类的意图；
如何在技术快速迭代的时代，建立人与机器之间可持续的信任关系。

Agent Harness 不是答案本身，而是我们寻找答案的方式。

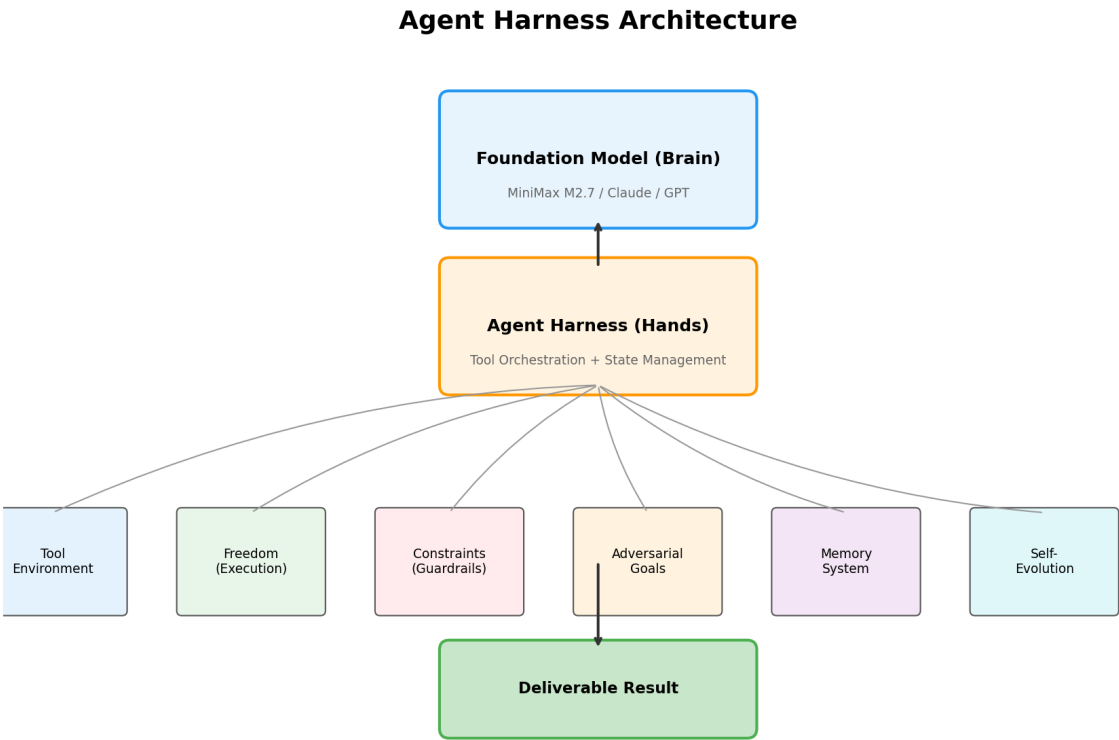


图 32: 全篇核心概念关联：Harness（能力）—Memory（关系）—Self-Evolution（进化）

27

²⁷ 视频画面时间区间：总结段。



图 33: 历史类比：从蒸汽机到电力到 AI，技术革命的社会影响曲线

28

本课程笔记完

愿每一位读者都能在 *Agent* 时代找到属于自己的 *Harness*

过好每一天，开心就好

²⁸ 视频画面时间区间：01:59:00–02:06:00。